

SolidaryTech

Documento de Especificação de Requisitos (Negócio + Técnico)

SRE, FinOps, Segurança/Disaster Recovery e ITSM/AIOps na nuvem Azure

PROJETO	SolidaryTech — Ecosistema de Microsserviços
CURSO	DevOps e Arquitetura Cloud
INTEGRANTES (NOME · RM · USERNAME)	Leonardo Alves Freitas · rm369434 · LndoaLves Antonio Demarchi · rm370045 · antoniodemarchi_73412
REPOSITÓRIOS	github.com/freitasleoalves/fiap-tc-5-* — 6 repositórios (\$1.2)
VÍDEO	[INSERIR LINK DO VÍDEO]
VERSÃO	1.5 — 26/09/2026

INFORMAÇÕES DO DOCUMENTO

Controle do documento

Versão	Data	Descrição
1.0	2026-09-25	Baseline de requisitos a partir do enunciado e da pasta Entrega/
1.1	2026-09-25	Reestruturação em 22 seções + anexos
1.2	2026-09-25	Auditoria de aderência ao código e correção dos documentos 01 a 05 (Anexo B)
1.3	2026-09-25	Identificação dos integrantes (nome, RM e username) e do curso, informada pelo grupo
1.4	2026-09-26	Validação integral contra o código. Removidas todas as afirmações sem comprovação no código: tempos de MTTR, valores monetários do forecast, relato de execução de restore e prazos de processo. Corrigidas duas imprecisões da 1.3 (senha do SonarQube e custo do Cosmos DB por tabela) e incluídas as divergências 11 e 12 do Anexo B.2 . Cada afirmação sobre a solução tem arquivo e linha no documento 06
1.5	2026-09-26	Rastreabilidade parte a parte: o documento 06 passou a seguir a ordem deste documento e indica, para cada linha de tabela, item, parágrafo e diagrama, onde está a confirmação no código

Regra de fonte

Toda afirmação deste documento sobre a solução implementada é **comprovada no código** da pasta **Entrega/**: arquivos dos 6 repositórios e seu histórico Git. O documento **06-RASTREABILIDADE-DOCUMENTACAO-CODIGO.md** segue a ordem deste documento e mostra, para cada linha de tabela, item, parágrafo e diagrama, o arquivo e a linha do código que o confirmam.

O documento contém apenas três tipos de conteúdo que não vêm do código, sempre identificados:

- **requisitos e regras do enunciado** *POSTECH - DCLT - Hackathon - Fase 5*, marcados com **[ENUNCIADO]** e a página;
- **identificação dos integrantes e do curso**, informada pelo grupo;
- **recomendações** desta especificação, marcadas com **[ANÁLISE]**, que não afirmam nada sobre o que está implementado.

Definições de negócio e de processo que não são configuração de código — SLA, forecast em valores monetários, processo de gestão de incidentes, papéis e procedimentos do PCN — estão nos documentos 01 a 05. Aqui elas aparecem apenas como **referência ao documento e à seção** onde estão definidas. Nenhum segredo é reproduzido.

Marcador	Significado
(sem marcador) ou [CÓDIGO]	Comprovado em arquivo dos repositórios (arquivo:linha no documento 06)
[GIT]	Comprovado no histórico Git dos repositórios
[ENUNCIADO]	Requisito ou regra do enunciado da coordenação
[ANÁLISE]	Recomendação desta especificação; não está implementado

CONTEÚDO

Sumário

SEÇÕES

01	Especificação do Projeto	4
02	Contexto e Problema	6
03	Escopo do Projeto	7
04	Demandas da Diretoria × Implementação	10
05	Stack de Tecnologias	12
06	Arquitetura Cloud na Azure	14
07	Microserviços e Instrumentação	17
08	Requisitos Funcionais, Não Funcionais e Regras de Negócio	19
09	Requisitos Técnicos do Enunciado (Frentes 0 a 4)	23
10	SRE — Confiabilidade e Golden Metrics	26
11	FinOps — Tagueamento, Custos e Rightsizing	29
12	ITSM e AIOps — Gestão de Incidentes	31
13	Segurança, PCN e Disaster Recovery	33
14	Estrutura dos Repositórios	35
15	Fluxos End-to-End	36
16	Estratégia de Validação	38
17	Entregáveis e Checklist do Enunciado	39
18	Matriz de Rastreabilidade	41
19	Riscos	42
20	Dificuldades e Lições Aprendidas	44
21	Questões em Aberto	46
22	Conclusão	47

ANEXOS

A	Mapeamento às Frentes e Escolhas de Ferramenta	48
B	Auditoria de Aderência ao Código	49
C	Inventário de Artefatos Analisados	52
D	Versões de Componentes	53
E	Glossário	54

SEÇÃO 1

Especificação do Projeto

1.1 Identificação

Campo	Valor	Base
Nome do Projeto	SolidaryTech — SRE, FinOps, Disaster Recovery e ITSM/AIOps (Fase 5 · Hackathon)	[ENUNCIADO]
Curso	DevOps e Arquitetura Cloud	Informado pelo grupo
Período dos commits	10/09/2026 a 25/09/2026	[GIT]
Disciplinas avaliadas	SRE, FinOps, Segurança e ITSM/AIOps, sobre a base das Fases 1 a 4	[ENUNCIADO] p. 2
Cloud Provider	Microsoft Azure: AKS, ACR, PostgreSQL Flexible, Service Bus, Cosmos DB, Storage	fiap-tc-5-terraform/*.tf
Regiões	eastus (padrão) · eastus2 (PostgreSQL)	variables.tf
Integrantes	Leonardo Alves Freitas (rm369434, Lndoaives) · Antonio Demarchi (rm370045, antoniodemarchi_73412)	Informado pelo grupo
GitHub	github.com/freitasleoalves	[GIT] remotes dos 6 repositórios

1.2 Continuidade e Repositórios da Entrega

A Fase 5 é um **projeto novo**: o código-base de `ngo-service`, `donation-service` e `volunteer-service` é fornecido pela coordenação ([ENUNCIADO] p. 2). O código registra que a entrega reaproveita os padrões validados nas Fases 3/4 do ToggleMaster:

- o Terraform segue o mesmo padrão, adaptado aos requisitos da Fase 5 (`fiap-tc-5-terraform/README.md`);
- o GitOps usa App-of-Apps + ApplicationSet + Kustomize base/overlays (`fiap-tc-5-gitops/README.md`);
- o remote state usa o mesmo storage, trocando só a `key` para `solidarytech.tfstate` (comentário do `providers.tf`).

O `notification-service` **não fazia parte do código original** e foi criado para fechar o trace distribuído assíncrono (`fiap-tc-5-notification-service/README.md`).

#	Repositório	Conteúdo	Linguagem / Stack
1	<code>fiap-tc-5-terraform</code>	AKS, ACR, rede, bancos, mensageria, backend do Velero, Datadog, PagerDuty	Terraform
2	<code>fiap-tc-5-gitops</code>	App-of-Apps, ApplicationSet, manifestos dos 4 apps, addons	Kustomize / YAML / Helm
3	<code>fiap-tc-5-donation-service</code>	Doações — Hot Path	Go 1.25
4	<code>fiap-tc-5-ngo-service</code>	Cadastro e listagem de ONGs	Python 3.11 / Flask
5	<code>fiap-tc-5-volunteer-service</code>	Voluntários vinculados a ONGs	Python 3.11 / Flask

#	Repositório	Conteúdo	Linguagem / Stack
6	fiap-tc-5-notification-service	Consumidor da fila de doações	Python 3.11 / Flask + worker

1.3 Documentos que Compõem a Entrega

Documento	Frente	Conteúdo
00-ESPECIFICACAO-DE-REQUISITOS.md	Todas	Este documento
01-SRE-SLI-SLO-SLA.md	Frente 1	Golden Metrics, 2 SLIs, SLOs, Error Budget, SLA, MTTR
02-FINOPS-FORECAST.md	Frente 2	Tagging, forecast, otimizações, rightsizing
03-PCN-DISASTER-RECOVERY.md	Frente 4	PCN, RPO/RTO, Velero, runbook, papéis
04-ITSM-INCIDENT-LIFECYCLE.md	Frente 3	Ciclo de vida do incidente, Watchdog, severidades, comunicação
05-PITCH-EXECUTIVO-ROTEIRO.md	Vídeo	Roteiro do pitch para a diretoria
06-RASTREABILIDADE-DOCUMENTACAO-CODIGO.md	Todas	Para cada parte deste documento, o arquivo e a linha do código que a confirmam

Os documentos 01 a 05 foram revisados na versão 1.2 para eliminar divergências com o código ([Anexo B.1](#)). A cópia anterior está em `fiap\_backup-docs-antes-auditoria-2026-09-25\`.

1.4 Convenções de Identificação

Prefixo	Significado
RN-	Requisito de negócio
RF-	Requisito funcional (domínio ou plataforma)
RGN- / BR-	Regra de negócio do domínio / regra de alerta
RNF-	Requisito não funcional
LIM-	Limitação do código-base
REQ-Fx.y	Requisito técnico do enunciado (Frente x, item y)
DEL- / EVD-	Entregável / evidência a produzir
RSK- / Q-	Risco ou lacuna / questão em aberto

Status de implementação: Implementado · Implementado c/ ressalva · Não evidenciado · Status de evidência: Texto (documentado em 01 a 05) · Pendente (print ou vídeo ausente na pasta).

SEÇÃO 2

Contexto e Problema

2.1 O Hackathon [ENUNCIADO]

O Tech Challenge da Fase 5 é um Hackathon obrigatório de 2 meses corridos, que vale 90% da nota das disciplinas da fase. A equipe precisa construir, orquestrar, monitorar, otimizar financeiramente e criar a estratégia de resiliência do ecossistema de microsserviços da SolidaryTech. A **regra de ouro** exige aplicar toda a base das Fases 1 a 4, sem deploy manual via `kubectl`, sem infraestrutura criada pelo console e com monitoramento profundo (p. 2).

2.2 O Problema [ENUNCIADO]

A SolidaryTech é uma iniciativa sem fins lucrativos que conecta ONGs a doadores e voluntários no Brasil. Depois de ganhar destaque em rede nacional, passou a ter picos de acesso imprevisíveis. A diretoria pede quatro garantias empresariais (p. 3), a primeira delas:

"Se a nuvem cair (Disaster Recovery), as doações não podem parar." — Enunciado, p. 3

As outras três são: custos tagueados e justificados; resposta a incidentes preditiva (AIOps); e SLO/SLA claros com as ONGs parceiras.

2.3 Dores Estruturais Atacadas

Dor [ENUNCIADO] p. 3	Resposta implementada no código	Artefato
D1 — Queda da nuvem não pode parar as doações	PITR do PostgreSQL + Velero (horário no <code>donation</code> , diário nos 4 namespaces de aplicação) + reconstrução por GitOps	<code>postgresql.tf</code> ; <code>addons/velero/schedule.yaml</code> ; <code>bootstrap/app-of-apps.yaml</code>
D2 — Custos fora de controle, precisam ser tagueados e justificados	Tags obrigatórias em código; o forecast está no documento 02	<code>main.tf</code>
D3 — Resposta a incidentes preditiva, não só reativa	Monitors de erro e latência → PagerDuty/Discord → self-heal; o Watchdog não tem configuração em código	<code>datadog.tf</code> ; <code>pagerduty.tf</code> ; <code>self-heal.yaml</code>
D4 — SLO/SLA claros com as ONGs	2 SLIs, SLOs e Error Budget no dashboard; o SLA está definido no documento 01 §5	<code>solidarytech-sre-dashboard.yaml</code>
D5 — Picos de acesso imprevisíveis	HPA 2→10 réplicas (CPU 70%) no <code>donation-service</code> ; AKS 3→5 nós	<code>hpa.yaml</code> ; <code>aks.tf</code> ; <code>variables.tf</code>

SEÇÃO 3

Escopo do Projeto

3.1 Visão do Projeto

A plataforma expõe três APIs de domínio (ONGs, doações e voluntários) e um consumidor assíncrono que **registra** no Cosmos DB cada evento de doação processado. O `notification-service` não envia mensagens a ONGs nem a voluntários: não há canal de entrega (e-mail, SMS ou push) no código (`notification-service/app.py`).

Disciplina	Exigência [ENUNCIADO]	Implementação no código	Documento
SRE	≥ 2 SLIs + SLO no <code>donation-service</code> ; dashboard de SLO e Error Budget; MTTR no relatório	Métricas em <code>otel.go</code> ; <code>solidarytech-sre-dashboard.yaml</code> ; <code>datadog.tf</code>	01
FinOps	Tags obrigatórias no Terraform; rightsizing via GitOps; forecast + recomendação	<code>main.tf</code> ; <code>apps/*/base/deployment.yaml</code> ; <code>cosmosdb.tf</code>	02
ITSM / AIOps	IA do APM ativada; ciclo de vida do incidente desenhado	<code>datadog.tf</code> ; <code>pagerduty.tf</code> ; <code>self-heal.yml</code>	04
Segurança / DR	PCN com RPO/RTO; DR prático (Opção A ou B)	<code>velero.tf</code> ; <code>addons/velero/</code> ; <code>postgresql.tf</code>	03

3.2 Objetivo

Atender às cinco frentes do enunciado com o que está nos 6 repositórios:

- infraestrutura Azure provisionada por Terraform; aplicações e addons entregues pelo ArgoCD;
- telemetria via OpenTelemetry para Prometheus, Loki e Datadog;
- SLO e Error Budget calculados no dashboard para o `donation-service`;
- tags de custo em código;
- backup agendado (Velero) e PITR;
- cadeia automática de alerta → incidente → self-heal para os serviços listados em `monitored_services` (padrão: `donation-service`).

3.3 As Cinco Frentes [ENUNCIADO] p. 3–5

- **Frente 0 (obrigatória):** comprovar Docker/Kubernetes, Terraform, CI/CD com DevSecOps, GitOps e Observabilidade/APM.
- **Frente 1:** a confiabilidade é declarada prioridade: SLIs, SLOs, dashboard de Error Budget e MTTR.
- **Frente 2:** o orçamento limitado da ONG exige tagging, rightsizing e forecast.
- **Frente 3:** prever incidentes antes de afetar o doador, com AIOps e fluxo de ITSM.
- **Frente 4:** a plataforma precisa sobreviver à queda do cluster principal, com PCN e DR prático.

3.4 Público-Alvo (Personas)

Persona	Necessidade	Base
Diretoria da SolidaryTech	Pitch de viabilidade: arquitetura, PCN e estratégias financeiras	[ENUNCIADO] p. 6
ONGs parceiras	SLO/SLA claros	[ENUNCIADO] p. 3
Doadores	Fluxo de doação (caminho crítico) disponível	[ENUNCIADO] p. 2–3

Persona	Necessidade	Base
Voluntários	Cadastro vinculado a uma ONG	<code>volunteer-service/app.py</code>
On-call	Incidente aberto no PagerDuty, com escalonamento em 10 min	<code>datadog.tf</code> ; <code>pagerduty.tf</code>
Squad de engenharia	Alerta no canal do Discord	<code>datadog.tf</code>
Avaliadores	Funcionamento demonstrado no vídeo e no relatório	[ENUNCIADO] p. 6–7

3.5 Riscos para o Negócio

Risco	Mitigação no código	Lacuna
Indisponibilidade do fluxo de doações	Monitors + self-heal + HPA	RSK-05
Perda de dados de doações	Retenção de backup de 14 d (PITR) + Velero horário	Mesma região (RSK-02)
Custo sem controle	Tags obrigatórias	RSK-15
Vazamento de credenciais	Os comentários do código preveem os valores reais aplicados no cluster via <code>kubectl</code> , fora do Git (<code>ignoreDifferences</code>)	Segredos reais versionados (RSK-01)
Exposição de dados pessoais	—	APIs sem autenticação (LIM-01, RSK-03)

3.6 Dentro e Fora do Escopo

Dentro: os 4 microsserviços; infraestrutura Azure; CI/CD com DevSecOps; GitOps; observabilidade + APM; SRE; FinOps; ITSM/AIOps; DR pela Opção A (Velero).

Fora (constatado no código):

Item	Constatação	Fonte
Gateway de pagamento real	Status <code>APPROVED</code> é simulação	<code>handlers.go</code> (comentário "Simulação de gateway de pagamento")
Ambiente espelho em outra região (Opção B)	Não implementado: o Terraform declara um único cluster AKS	<code>aks.tf</code>
Reserved Instances / Savings Plan	Não aplicado: nenhum recurso de reserva no Terraform (recomendação no documento 02 §4)	<code>*.tf</code>
SLO e Monitors para ngo, volunteer e notification	Não configurados: o dashboard SRE e o padrão de <code>monitored_services</code> cobrem só o <code>donation-service</code>	Dashboard SRE; <code>variables.tf</code>
Rotas do código original	Mantidas como estão	Comentários em <code>apps/*/overlays/prod/ingress.yaml</code>

Inexistente na pasta: frontend, autenticação de APIs, entidade Campanha, canal de entrega de notificações, ambientes além de `overlays/prod` e testes de carga.

3.7 Premissas, Restrições e Dependências

Tipo	ID	Descrição	Fonte
Premissa	P-01	Ambiente único de produção (<code>Environment = "Production"</code> , <code>environment = "prod"</code>)	<code>main.tf</code> ; <code>variables.tf</code>
Restrição	C-01	Sem deploy manual, sem console, sem voo cego	[ENUNCIADO] p. 2
Restrição	C-02	Nós <code>Standard_B2s</code> , mínimo de 3 (lição registrada da Fase 4)	<code>variables.tf</code> ; <code>aks.tf</code>
Restrição	C-03	Vídeo de até 20 min	[ENUNCIADO] p. 6
Restrição	C-04	Bancos, mensageria e backend do Velero só existem com <code>deploy_databases = true</code> (padrão <code>false</code>)	<code>variables.tf</code>
Dependência	X-01	Storage do remote state pré-existente (<code>sttfstatebsouth</code>), fora deste código	<code>providers.tf</code>
Dependência	X-02	Contas Datadog (<code>datadoghq.com</code>) e PagerDuty; webhook do Discord	<code>variables.tf</code> ; <code>datadog.tf</code>
Dependência	X-03	Tokens do GitHub: <code>GITOPS_TOKEN</code> e <code>github_selfheal_token</code>	<code>build-push.yaml</code> ; <code>variables.tf</code>

SEÇÃO 4

Demandas da Diretoria × Implementação

4.1 Visão Geral

Demanda [ENUNCIADO]	O que o código implementa	Situação
Doações não param se a nuvem cair	Retenção de backup de 14 d no <code>donation_db</code> (PITR), Velero horário e diário, reconstrução do cluster via Terraform + bootstrap do ArgoCD	Implementado c/ ressalva backups na mesma região, sem cobertura de perda de região (RSK-02)
Custos tagueados e justificados	9 recursos Azure com <code>Project</code> , <code>Environment</code> , <code>CostCenter</code> e <code>Component</code> ; forecast no documento 02	Evidência visual pendente
Resposta preditiva (AIOps)	Monitors de erro 5xx e de latência p95; o Watchdog não tem configuração em código	Watchdog sem evidência (RSK-04)
SLO/SLA com as ONGs	SLOs no dashboard (99,9% e p95 ≤ 500 ms); SLA definido no documento 01 §5	Implementado (SLO)
Picos imprevisíveis	HPA 2→10 no <code>donation-service</code> ; autoscaling de nós 3→5	Ver RSK-05

4.2 Caminho Crítico com Tratamento Diferenciado

O `donation-service` é o único serviço com:

- SLO no dashboard (`solidarytech-sre-dashboard.yaml`);
- HPA (`apps/donation/overlays/prod/hpa.yaml`);
- retenção de backup de 14 d e tag `Criticality = "hot-path"` (`postgresql.tf`);
- Schedule horário do Velero (`schedule.yaml`);
- os maiores limits de CPU e memória: 500m/256Mi (`deployment.yaml`).

4.3 Cadeia Automática de Resposta a Incidentes

O código configura uma cadeia automática. Quando um Monitor do Datadog entra em Alert, ele abre incidente no PagerDuty e notifica o Discord; o Monitor de erro 5xx também dispara `repository_dispatch`, e o workflow `self-heal.yml` executa `kubectl rollout restart` no serviço (`datadog.tf`, `pagerduty.tf`, `self-heal.yml`). A pasta **não contém medição do tempo** dessa cadeia no ambiente SolidaryTech (RSK-04, EVD-10).

4.4 Cosmos DB em Serverless

- `cosmosdb.tf` declara `EnableServerless`, e as tabelas não declaram `throughput`.
- O repositório nasce assim no commit inicial `82421a8` [GIT].
- O comentário do próprio `cosmosdb.tf` compara as duas configurações: com 400 RU/s provisionados por tabela (800 RU/s no total), o custo seria fixo em ~US\$ 46,72/mês, independente do uso; no modo serverless, a cobrança é de US\$ 0,25 por milhão de RUs consumidas.

4.5 Ambiente Reconstruível por Código

O ArgoCD é instalado pelo Terraform (`argocd.tf`). O README do GitOps descreve o bootstrap com `kubectl apply -f bootstrap/app-of-apps.yaml`, aplicado uma única vez, e registra: "A partir daí o ArgoCD assume o resto sozinho." Os Schedules do Velero estão versionados em `addons/velero/schedule.yaml`.

4.6 Rastreabilidade Código → Produção

As tags de imagem nos overlays de produção (`09f6e6e`, `73d5d36`, `87ae20a`, `5fafa72`) são iguais ao HEAD de cada repositório de serviço `[GIT]`. O pipeline grava cada deploy como commit `deploy: update <svc> to <sha>` no repositório GitOps (`build-push.yaml`).

SEÇÃO 5

Stack de Tecnologias

Para cada tecnologia: o papel no projeto e a implementação, conforme o código e seus comentários.

#	Tecnologia	Papel no projeto	Implementação	Fonte
5.1	Terraform	Provisionar a infraestrutura (rede, cluster, registry, bancos, mensageria, backend do Velero, ArgoCD, alertas) e aplicar as tags obrigatórias	14 arquivos <code>.tf</code> ; providers <code>azurerm</code> ~> 5.5, <code>azuread</code> ~> 3.9, <code>helm</code> ~> 3.3, <code>datadog</code> ~> 4.21, <code>pagerduty</code> ~> 3.36; backend <code>azurerm</code> ; flag <code>deploy_databases</code>	<code>providers.tf</code> ; <code>variables.tf</code>
5.2	AKS + ACR	Executar os serviços e addons; registry das imagens	AKS B2s com autoscaling 3–5, Azure CNI + Calico, OIDC issuer, identidade SystemAssigned; ACR Basic com role <code>AcrPull</code> para o kubelet	<code>aks.tf</code>
5.3	PostgreSQL Flexible	Dados de <code>ngo-service</code> e <code>donation-service</code>	2 servidores v15, <code>B_Standard_B1ms</code> , 32 GB, em <code>eastus2</code> ; retenção de 7 d (ngo) e 14 d (donation), o que habilita PITR segundo o comentário do arquivo	<code>postgresql.tf</code>
5.4	Service Bus	Desacoplar doação e notificação	Namespace Basic; fila <code>donation-events</code> (<code>max_delivery_count</code> = 10, lock de 30 s); substitui o SQS do código original, ainda aceito via <code>CLOUD_PROVIDER</code>	<code>servicebus.tf</code> ; <code>main.go</code>
5.5	Cosmos DB Table API	Tabelas <code>Volunteers</code> e <code>DonationNotifications</code>	Conta Serverless, consistência Session, 1 região; substitui o DynamoDB do código original. Trade-off registrado: teto de 5.000 RU/s, 50 GB por container, sem geo-replicação	<code>cosmosdb.tf</code>
5.6	GitHub Actions — CI/CD	Validar, escanear, construir e publicar as imagens; atualizar o GitOps	5 jobs: <code>lint</code> (golanci-lint v2.12 / flake8), <code>test</code> (go test / pytest + cobertura), <code>sonarqube</code> , <code>build-scan-push</code> (Trivy FS → build → Trivy imagem → push no ACR com as tags <code>\${{ github.sha }}</code> e <code>latest</code>), <code>update-gitops</code>	<code>build-push.yaml</code>
5.7	ArgoCD + Kustomize	Git como fonte do estado do cluster	Chart 9.4.15 via Terraform; <code>bootstrap/app-of-apps.yaml</code> → <code>applicationsets/</code> (Application <code>cluster-addons</code> + ApplicationSet <code>apps</code> sobre <code>apps/*/overlays/prod</code>); todas com <code>automated: selfHeal + prune</code> ; <code>CreateNamespace=true</code> no ApplicationSet e nas Applications de addon	<code>argocd.tf</code> ; <code>bootstrap/</code> ; <code>applications</code> <code>ets/</code> ; <code>addons/*/application.yaml</code>
5.8	Ingress NGINX	Entrada HTTP	Chart 4.12.1, Service LoadBalancer, probe <code>/healthz</code> ; rotas <code>/ngos</code> , <code>/donations</code> , <code>/volunteers</code> , <code>/grafana</code> , <code>/</code> (SonarQube)	<code>argocd.tf</code> ; <code>*ingress.yaml</code>
5.9	OpenTelemetry Collector	"Peça central de telemetria" dos 4 serviços (comentário do values)	Chart 0.173.0, imagem contrib 0.160.0. Gateway (Deployment, OTLP :4317/:4318): métricas → Prometheus, logs → Loki, traces → Datadog. Logs (DaemonSet, <code>/var/log/pods</code>) → Loki	<code>addons/otel-collector-*/values.yaml</code>
5.10	kube-prometheus-stack	Métricas, dashboards e alertas de infraestrutura	Chart 90.0.0; Prometheus com remote-write receiver, retenção de 30 d , 20 Gi; Grafana fixado em 11.4.0 (estratégia <code>Recreate</code> , sidecar de dashboards, datasource Loki, <code>/grafana</code>); Alertmanager ativo	<code>addons/kube-prometheus-stack/values.yaml</code>

#	Tecnologia	Papel no projeto	Implementação	Fonte
5.11	Loki	Logs centralizados	Chart 7.3.0 SingleBinary; filesystem de 10 Gi; retenção de 72 h ; <code>chunksCache</code> desligado (o padrão do chart pede ~9,6 Gi de memória, incompatível com nós <code>Standard_B2s</code>)	<code>addons/loki/values.yaml</code>
5.12	Datadog	APM, Service Map, Monitors	Exporter <code>datadog</code> do Collector (site <code>datadoghq.com</code>) com feature gate <code>- exporter.datadogexporter.DisableAPMStats</code> para manter as métricas <code>trace.*</code> ; Monitors em Terraform; o Watchdog não tem resource (automático, segundo o comentário do <code>datadog.tf</code>)	<code>otel-collector-gateway/values.yaml</code> ; <code>datadog.tf</code>
5.13	PagerDuty	Incidentes com escalonamento	Escalation Policy "SolidaryTech - On-call" (10 min, 2 loops, 1 usuário) + Service <code>solidarytech-<svc></code> com integração Datadog	<code>pagerduty.tf</code>
5.14	Discord	ChatOps	Webhook <code>discord-alerts</code> (sufixo <code>/slack</code>) com título, estado, prioridade e link	<code>datadog.tf</code>
5.15	GitHub Actions — Self-Healing	Mitigação automática	<code>self-heal.yaml</code> em cada repositório de serviço, via <code>repository_dispatch</code> (<code>self-heal</code>) ou <code>workflow_dispatch</code> ; SP dedicado criado pelo Terraform	<code>self-heal.yaml</code> ; <code>github_actions.tf</code>
5.16	Velero	Backup do estado do cluster para fora do cluster: manifestos no Blob Storage e volumes por snapshot de Azure Disk	Backend em <code>velero.tf</code> ; chart 12.1.0 + plugin Azure v1.13.1 + CSI + node agent + 2 Schedules	<code>velero.tf</code> ; <code>addons/velero/</code>

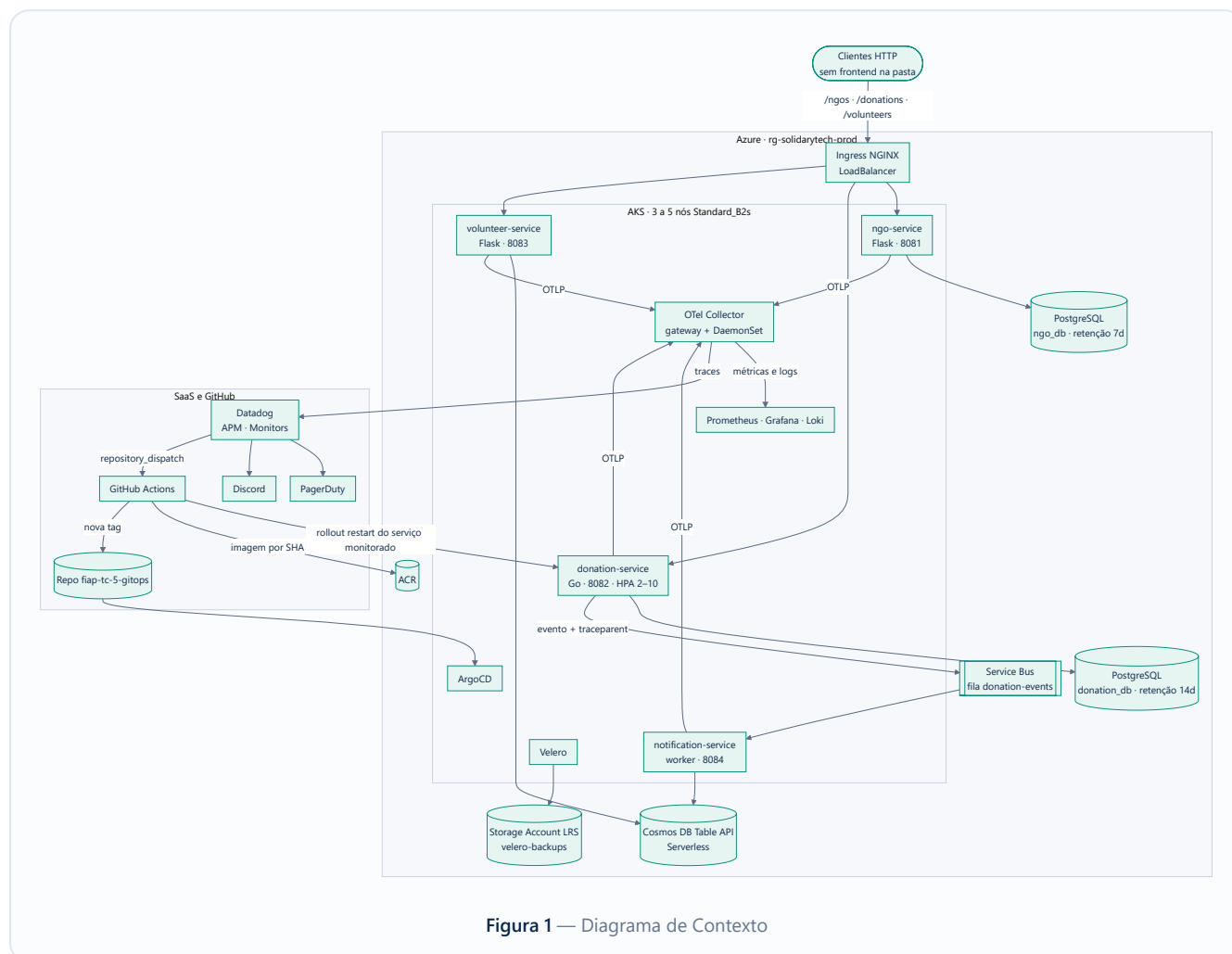
Justificativas registradas no código:

- 5.2 — mínimo de 3 nós: na Fase 4, nós B2s sob pressão de memória derrubaram o kubelet (`aks.tf`).
- 5.5 — Serverless em vez de 800 RU/s fixos (`cosmosdb.tf`).
- 5.10 — retenção de 30 d para o dashboard SRE calcular o Error Budget mensal sem depender só do Datadog, que é pago por volume; Grafana 11.4.0 porque a 13.x quebrava os datasources Prometheus/Loki (`values.yaml`).
- 5.12 — APM Stats reativado para manter Service Map e Monitors com um único exporter (`values.yaml`).

SEÇÃO 6

Arquitetura Cloud na Azure

6.1 Diagrama de Contexto



6.2 Fluxo de Telemetria

```

Microserviços (1 Go + 3 Python)
    |
    | OTLP gRPC :4317 (Go) / HTTP-protobuf :4318 (Python)
    v
otel-collector (Deployment · memory_limiter → k8sattributes → batch)
    ├── métricas → prometheusremotewrite → Prometheus (retenção 30 d)
    ├── logs      → otlphttp/loki      → Loki (retenção 72 h)
    └── traces    → exporter datadog    → Datadog

Todos os pods do cluster — filelog (/var/log/pods) → otel-collector-logs (DaemonSet) → Loki
Prometheus + Loki → Grafana → Ingress NGINX → /grafana

```

TEXTO

Fonte: `addons/otel-collector-gateway/values.yaml`, `addons/otel-collector-logs/values.yaml`, `apps/*/base/deployment.yaml`.

6.3 Componentes Provisionados via Terraform

Recurso	Configuração relevante	Tag Component	Arquivo
Resource Group <code>rg-solidarytech-prod</code>	—	<code>core</code>	<code>main.tf</code>
VNet 10.0.0.0/16 · Subnet 10.0.0.0/20	Subnet sem tags	<code>network</code> (VNet)	<code>network.tf</code>
ACR Basic	<code>admin_enabled = true</code>	<code>acr</code>	<code>aks.tf</code>
AKS <code>aks-solidarytech-prod</code>	B2s, 3–5 nós, Azure CNI + Calico, service CIDR 172.16.0.0/16, OIDC issuer	<code>aks</code>	<code>aks.tf</code>
PostgreSQL Flexible ×2 (<code>eastus2</code>)	v15, B1ms, 32 GB, retenção 7/14 d, <code>geo_redundant_backup_enabled = false</code> , zona 1, acesso público + regra <code>AllowAzureServices</code>	<code>ngo-service</code> / <code>donation-service</code> (+ <code>Criticality</code>)	<code>postgresql.tf</code>
Service Bus Basic + fila <code>donation-events</code>	<code>max_delivery_count = 10</code> , lock 30 s	<code>messaging</code>	<code>servicebus.tf</code>
Cosmos DB Table API	Serverless, Session, 1 <code>geo_location</code>	<code>nosql</code>	<code>cosmosdb.tf</code>
Storage do Velero + container <code>velero-backups</code>	Standard LRS, TLS 1.2, container privado	<code>disaster-recovery</code>	<code>velero.tf</code>
SP do Velero	<code>Storage Account Contributor</code> no storage + <code>Contributor</code> no node resource group	—	<code>velero.tf</code>
SP do self-healing	Azure Kubernetes Service Cluster Admin Role no AKS	—	<code>github_actions.tf</code>
Helm: ArgoCD 9.4.15 · ingress-nginx 4.12.1	ArgoCD <code>server.service.type = LoadBalancer</code>	—	<code>argocd.tf</code>
Datadog	2 Monitors por serviço monitorado, webhooks para Discord e self-heal, integração PagerDuty	—	<code>datadog.tf</code>
PagerDuty	Escalation Policy + Service por serviço monitorado	—	<code>pagerduty.tf</code>

6.4 Addons Entregues via GitOps

Application	Chart / versão	Namespace	Função	Requests → Limits
<code>addon-kube-prometheus-stack</code>	<code>prometheus-community</code> 90.0.0	<code>monitoring</code>	Prometheus (20 Gi) + Grafana (2 Gi) + Alertmanager (1 Gi) + node-exporter + kube-state-metrics	Prometheus 150m/512Mi → 500m/1Gi · Grafana 100m/384Mi → 500m/768Mi · Alertmanager 25m/64Mi → 100m/128Mi
<code>addon-loki</code>	<code>grafana/loki</code> 7.3.0	<code>monitoring</code>	Logs	100m/256Mi → 500m/512Mi
<code>addon-otel-collector-gateway</code>	<code>opentelemetry-collector</code> 0.173.0	<code>monitoring</code>	Gateway OTLP	100m/200Mi → 500m/400Mi
<code>addon-otel-collector-logs</code>	<code>opentelemetry-collector</code> 0.173.0	<code>monitoring</code>	DaemonSet de logs	50m/100Mi → 200m/200Mi

Application	Chart / versão	Namespace	Função	Requests → Limits
addon-grafana-dashboards	— (ConfigMaps do diretório addons/grafana-dashboards)	monitoring	Dashboards Overview e SRE	—
addon-sonarqube	SonarSource 2026.4.1 (community)	sonarqube	SAST (5 Gi + PostgreSQL embutido de 5 Gi)	400m/1Gi → 1/2Gi
addon-velero	vmware-tanzu 12.1.0	velero	Backup e restore	100m/128Mi → 500m/256Mi

Os addons com chart usam Applications **multi-source** (chart upstream + `values.yaml` do repositório GitOps, e em alguns casos um diretório extra com secrets, ingress ou schedule).

6.5 Namespaces e Exposição Externa

Namespace	Conteúdo	Exposição
ngo · donation · volunteer	Serviços de domínio	Ingress <code>/ngos</code> , <code>/donations</code> , <code>/volunteers</code> , sem bloco <code>tls</code>
notification	Consumidor da fila	Sem Ingress
monitoring	Prometheus, Grafana, Loki, Collectors	Único Ingress: Grafana (<code>/grafana</code>), sem <code>tls</code>
sonarqube	SonarQube	Ingress no path <code>/</code> , sem <code>tls</code>
velero	Velero	Sem Ingress
argocd	ArgoCD	Service <code>LoadBalancer</code> próprio
ingress-nginx	Controller	Service <code>LoadBalancer</code>

A network policy Calico está habilitada no AKS, mas não existe nenhum manifesto `NetworkPolicy` no repositório GitOps.

SEÇÃO 7

Microsserviços e Instrumentação

7.1 Catálogo de Serviços

Serviço	Responsabilidade	Stack	Porta	Persistência	Réplicas	Requests → Limits
ngo-service	Cadastro e listagem de ONGs	Python 3.11 · Flask 2.2.2 · gunicorn 20.1.0	8081	PostgreSQL ngo_db	2	50m/64Mi → 200m/128Mi
donation-service	Doações (Hot Path)	Go 1.25	8082	PostgreSQL donation_db + Service Bus	2 (HPA 2-10)	100m/64Mi → 500m/256Mi
volunteer-service	Voluntários por ONG	Python 3.11 · Flask	8083	Cosmos Volunteers	2	50m/96Mi → 250m/192Mi
notification-service	Registro dos eventos de doação	Python 3.11 · Flask + thread worker	8084	Service Bus → Cosmos DonationNotifications	2	50m/64Mi → 200m/128Mi

7.2 Instrumentação OpenTelemetry

Serviço	Modo	Endpoint OTLP	Protocolo	OTEL_LOGS_EXPORTER	Métricas customizadas
donation-service	SDK manual (otel.go) + otelhttp	otel-collector.monitoring...:4317	grpc	não definido	solidarytech_http_requests_total, solidarytech_http_request_duration_seconds (buckets de 5 ms a 10 s)
ngo-service	opentelemetry-instrument + meter manual	...:4318	http/protobuf	none	as duas acima
volunteer-service	idem	...:4318	http/protobuf	none	as duas acima
notification-service	idem + tracer/meter manuais no worker	...:4318	http/protobuf	none	as duas acima + solidarytech_messages_processed_total{status}

Todos definem `OTEL_SERVICE_NAME` e `OTEL_RESOURCE_ATTRIBUTES=deployment.environment=production`; os serviços Python também definem `OTEL_PYTHON_LOG_CORRELATION=true` (`apps/*/base/deployment.yaml`).

Escopo das métricas: tanto o middleware Go (`withMetrics`) envolvendo o roteador inteiro, `main.go`) quanto os hooks Flask (`after_request`) registram **todas as rotas**, inclusive `/health` .

7.3 Padrão Go vs. Python

- Go:** `init0Tel` configura TracerProvider e MeterProvider com OTLP/gRPC e o propagador W3C TraceContext + Baggage (`otel.go`).
- Python:** a imagem executa `opentelemetry-bootstrap -a install` e sobe com `opentelemetry-instrument gunicorn` , sem `--preload` . O comentário no `Dockerfile` do ngo e do volunteer cita uma lição da Fase 4.

7.4 Rastreamento Distribuído Assíncrono

1. O `donation-service` injeta `traceparent / tracestate` nas `ApplicationProperties` da mensagem do Service Bus (`servicebus.go`); no modo SQS, usa `MessageAttributes` (`sqs.go`).
2. O envio roda em goroutine com o `SpanContext` da requisição preservado (`handlers.go`).
3. O `notification-service` extrai o contexto e abre o span `notification.process_event` (`SpanKind.CONSUMER`) com os atributos `messaging.system`, `solidarytech.donation_id` e `solidarytech.ngo_id` (`app.py`).

Não há chamadas HTTP entre serviços no código: essa fila é o único elo entre serviços.

7.5 Modelo de Dados

Armazenamento	Entidade	Campos	Chaves / restrições	Fonte
PostgreSQL <code>ngo_db</code>	<code>ngos</code>	<code>id</code> , <code>name</code> (150), <code>email</code> (100), <code>cause</code> (100), <code>city</code> (100), <code>created_at</code>	PK <code>id</code> ; <code>email</code> UNIQUE	<code>ngo-service/db/init.sql</code>
PostgreSQL <code>donation_db</code>	<code>donations</code>	<code>id</code> , <code>ngo_id</code> , <code>amount</code> NUMERIC(10, 2), <code>donor_name</code> (100), <code>status</code> (20), <code>created_at</code>	PK <code>id</code>	<code>donation-service/db/init.sql</code>
Cosmos <code>Volunteers</code>	voluntário	<code>name</code> , <code>email</code> , <code>ngo_id</code> , <code>registered_at</code>	<code>PartitionKey = ngo_id</code> , <code>RowKey = volunteer_id</code> (UUID)	<code>volunteer-service/app.py</code>
Cosmos <code>DonationNotifications</code>	registro de notificação	<code>donation_id</code> , <code>ngo_id</code> , <code>amount</code> , <code>donor_name</code> , <code>status</code> , <code>processed_at</code>	<code>PartitionKey = ngo_id</code> , <code>RowKey = UUID</code>	<code>notification-service/app.py</code>

O schema do PostgreSQL é criado pelos Jobs `db-init-ngo` e `db-init-donation`, a partir de ConfigMaps nos overlays de produção.

SEÇÃO 8

Requisitos Funcionais, Não Funcionais e Regras de Negócio

8.1 Requisitos de Negócio (RN)

ID	Requisito	Origem	Como é atendido no código	Status
RN-01	Doações continuam mesmo com falha da nuvem	[ENUNCIADO] p. 3	PITR, Velero, reconstrução GitOps	Implementado c/ ressalva (RSK-02)
RN-02	Custos taguados e justificados	[ENUNCIADO] p. 3	Tags em código; forecast no documento 02	Implementado Evidência pendente
RN-03	Resposta a incidentes preditiva (AIOps)	[ENUNCIADO] p. 3	Sem configuração de Watchdog em código	Não evidenciado
RN-04	SLO/SLA claros com as ONGs	[ENUNCIADO] p. 3	SLOs no dashboard; SLA definido no documento 01 §5	Implementado
RN-05	Suportar picos imprevisíveis	[ENUNCIADO] p. 3	HPA + autoscaling do AKS	Implementado c/ ressalva (RSK-05)
RN-06	Doações como Caminho Crítico	[ENUNCIADO] p. 2	SLO, HPA, backup horário, retenção de 14 d, Criticality = "hot-path"	Implementado
RN-07	Pitch de viabilidade à diretoria	[ENUNCIADO] p. 6	Roteiro no documento 05	Vídeo pendente
RN-08	Base das Fases 1 a 4 sem deploy manual nem console	[ENUNCIADO] p. 2	Frente 0 (§9.1)	Implementado c/ ressalva (RSK-06, RSK-07)

8.2 Requisitos Funcionais de Domínio (como implementados)

ID	O sistema deve...	Respostas	Fonte
RF-01	cadastrar ONG com <code>name</code> , <code>email</code> , <code>cause</code> e <code>city</code> obrigatórios (<code>POST /ngos</code>)	201 · 400 · 409 (e-mail duplicado) · 500	<code>ngo-service/app.py</code>
RF-02	listar ONGs em ordem decrescente de <code>id</code> (<code>GET /ngos</code>)	200 · 500	<code>ngo-service/app.py</code>
RF-03	registrar doação com <code>ngo_id</code> , <code>amount</code> e <code>donor_name</code> , atribuindo <code>status = APPROVED</code> (<code>POST /donations</code>)	201 · 400 (JSON inválido) · 500	<code>handlers.go</code>
RF-04	publicar um evento por doação aprovada, com contexto de trace, quando a mensageria está configurada	—	<code>handlers.go</code> ; <code>servicebus.go</code>
RF-05	listar doações (<code>GET /donations</code>) e rejeitar outros métodos	200 · 405 · 500	<code>handlers.go</code>
RF-06	registrar voluntário vinculado a uma ONG (<code>POST /volunteers</code>), gerando UUID e <code>registered_at</code>	201 · 400 · 500	<code>volunteer-service/app.py</code>
RF-07	listar voluntários de uma ONG (<code>GET /volunteers/<int:ngo_id></code>)	200 · 500	<code>volunteer-service/app.py</code>

ID	O sistema deve...	Respostas	Fonte
RF-08	consumir continuamente a fila <code>donation-events</code> e registrar cada evento no Cosmos	—	<code>notification-service/app.py</code>
RF-09	expor <code>GET /health</code> em todos os serviços	200	<code>app.py</code> ; <code>handlers.go</code>

8.3 Requisitos Funcionais de Plataforma

ID	A plataforma deve...	Status	Fonte
RF-10	provisionar o ambiente via Terraform com as tags obrigatórias (<code>local.mandatory_tags</code> , aplicado em 9 recursos)	Implementado	<code>main.tf</code> ; <code>*.tf</code>
RF-11	reprovar o pipeline quando o Trivy (FS ou imagem) encontrar CVE <code>CRITICAL</code> com correção disponível	Implementado	<code>build-push.yaml</code> (<code>exit-code: "1"</code> , <code>ignore-unfixed: "true"</code>)
RF-12	publicar imagens por SHA e registrar cada deploy como commit no GitOps	Implementado	<code>build-push.yaml</code>
RF-13	descobrir e sincronizar todo app sob <code>apps/*/overlays/prod</code>	Implementado	<code>apps-appset.yaml</code>
RF-14	calcular os SLIs de erro e latência do <code>donation-service</code> e o Error Budget restante	Implementado	<code>solidarytech-sre-dashboard.yaml</code>
RF-15	abrir incidente e notificar a squad quando um Monitor entrar em Alert	Implementado	<code>datadog.tf</code> ; <code>pagerduty.tf</code>
RF-16	executar <code>rollout restart</code> no serviço quando o Monitor de erro 5xx disparar	Implementado	<code>datadog.tf</code> ; <code>self-heal.yml</code>
RF-17	detectar anomalias sem regra manual (AIOps)	Não evidenciado	<code>datadog.tf</code> (sem resource para o Watchdog)
RF-18	fazer backup agendado (horário no <code>donation</code> , diário nos 4 namespaces de aplicação)	Implementado c/ ressalva	aplicação dos Schedules pelo ArgoCD não evidenciada na pasta (RSK-07)
RF-19	permitir restore por namespace (Velero) e PITR do banco	Implementado (configuração); execução sem evidência na pasta (EVD-08)	<code>velero.tf</code> ; <code>addons/velero/</code> ; <code>postgresql.tf</code>

8.4 Regras de Negócio e Regras de Alerta

ID	Regra	Fonte
RGN-01	Toda doação criada recebe <code>status = APPROVED</code> (simulação de gateway)	<code>handlers.go</code>
RGN-02	O e-mail da ONG é único	<code>init.sql</code> + 409 em <code>app.py</code>
RGN-03	Valor da doação em <code>NUMERIC(10, 2)</code>	<code>donation-service/db/init.sql</code>
RGN-04	Voluntário sempre vinculado a uma ONG (<code>PartitionKey = ngo_id</code>)	<code>volunteer-service/app.py</code>

ID	Regra	Fonte
RGN-05	Sem mensageria configurada, a doação é gravada e só o log <code>[QUEUE_DISABLED]</code> é registrado	<code>handlers.go</code>
RGN-06	Carga inicial com 2 ONGs (Anjos de Patas — Osasco; Educa Mais — São Paulo)	<code>apps/ngo/overlays/prod/configmap.yaml</code>
BR-01	Taxa de 5xx > 5% em 5 min → PagerDuty + Discord + self-heal; renotifica a cada 10 min	<code>datadog.tf</code> (<code>http_5xx_error_rate</code>)
BR-02	Latência p95 > 500 ms em 5 min → PagerDuty + Discord (sem self-heal)	<code>datadog.tf</code> (<code>latency_p95</code>)

8.5 Requisitos Não Funcionais (RNF)

ID	Categoria	Meta / valor	Fonte
RNF-01	Disponibilidade (SLA externo)	Definido no documento 01 §5 (≥ 99,5% de disponibilidade mensal do fluxo de doações)	Documento 01 §5
RNF-02	Confiabilidade (SLO)	≥ 99,9% de requisições não-5xx em 30 d	Dashboard SRE
RNF-03	Desempenho (SLO)	p95 ≤ 500 ms (janela de 5 min)	Dashboard SRE; <code>datadog.tf</code>
RNF-04	Escalabilidade	HPA 2–10 réplicas (CPU 70%); AKS 3–5 nós	<code>hpa.yaml</code> ; <code>variables.tf</code>
RNF-05	Recuperabilidade	Doações: RPO ≤ 5 min / RTO ≤ 30 min · demais: RPO ≤ 15 min / RTO ≤ 2 h	<code>terraform/README.md</code>
RNF-06	Backup	Velero horário (TTL 7 d) + diário 02:00 (TTL 30 d); retenção PostgreSQL 14 d / 7 d	<code>schedule.yaml</code> ; <code>postgresql.tf</code>
RNF-07	Observabilidade	Métricas 30 d; logs 72 h; traces no Datadog; W3C trace context na fila	values dos addons; <code>otel.go</code>
RNF-08	Deteção	Monitors avaliam janelas de 5 min (<code>sum(last_5m)</code> e <code>avg(last_5m)</code>)	<code>datadog.tf</code>
RNF-09	Escalonamento	PagerDuty escala em 10 min, com 2 loops; os demais prazos de atendimento estão nos documentos 01 §5 e 04 §2	<code>pagerduty.tf</code>
RNF-10	DevSecOps	Trivy FS + imagem bloqueando CRITICAL com correção; SonarQube; lint	<code>build-push.yaml</code>
RNF-11	Imutabilidade	Overlays referenciam a imagem por SHA	<code>kustomization.yaml</code>
RNF-12	Criptografia em trânsito	PostgreSQL <code>sslmode=require</code> ; Storage TLS 1.2; Cosmos HTTPS	<code>outputs.tf</code> ; <code>velero.tf</code>
RNF-13	FinOps	Tags em código; forecast no documento 02	<code>main.tf</code>
RNF-14	Extensibilidade	Nova pasta <code>apps/<app>/</code> descoberta automaticamente	<code>fiap-tc-5-gitops/README.md</code>

8.6 Limitações do Código-Base (backlog)

A limitação é comprovada no código; a coluna "Impacto" é leitura desta especificação **[ANÁLISE]**.

ID	Limitação	Impacto	Fonte
LIM-01	Nenhuma API tem autenticação. <code>GET /donations</code> expõe <code>donor_name</code> e <code>GET /volunteers/<int:ngo_id></code> expõe nome e e-mail pelo Ingress	Dados pessoais expostos (LGPD)	<code>handlers.go</code> ; <code>app.py</code> ; <code>ingress.yaml</code>
LIM-02	<code>POST /donations</code> não valida valor, existência de <code>ngo_id</code> nem <code>donor_name</code>	Dado inconsistente	<code>handlers.go</code>
LIM-03	Não há idempotência: um reenvio duplica a doação	Duplicidade	<code>handlers.go</code>
LIM-04	O evento é enviado em goroutine depois do INSERT; a falha só gera log, sem retry	Evento perdido	<code>handlers.go</code>
LIM-05	O worker chama <code>complete_message</code> mesmo quando <code>process_message</code> captura erro	Falhas tratadas não geram reentrega nem dead-letter	<code>notification-service/app.py</code>
LIM-06	Listagens sem paginação	Latência cresce com o volume	<code>handlers.go</code> ; <code>app.py</code>
LIM-07	ONGs: só cadastro e listagem; voluntários: só vínculo com ONG (sem entidade Campanha)	Diferença em relação ao enunciado (p. 2)	<code>app.py</code>
LIM-08	Um sender do Service Bus é criado por mensagem	Overhead por doação	<code>servicebus.go</code>
LIM-09	O <code>notification-service</code> só registra; não entrega notificação a ONG nem a voluntário	Escopo funcional limitado	<code>notification-service/app.py</code>

8.7 Critérios de Aceite do Caminho Crítico (BDD)

[ANÁLISE] Critérios derivados do comportamento do código, a serem verificados em execução.

Funcionalidade: Processamento de doações (Hot Path)

GHERKIN

Cenário: Doação válida é aprovada, persistida e registrada
 Dado que o `donation-service` está com `CLOUD_PROVIDER=azure`
 Quando o cliente envia `POST /donations` com `ngo_id`, `amount` e `donor_name`
 Então a resposta é 201 com status "APPROVED", `id` e `created_at`
 E um evento é publicado na fila "donation-events" com `traceparent`
 E o `notification-service` grava o registro em "DonationNotifications"

Cenário: Falha de banco consome Error Budget
 Dado que o PostgreSQL `donation_db` está indisponível
 Quando o cliente envia `POST /donations`
 Então a resposta é 500
 E `solidarytech_http_requests_total{http_status_code="500"}` é incrementado
 E o painel "Error Budget Restante (30d)" diminui

Cenário: Taxa de erro alta dispara a cadeia de incidente (BR-01)
 Dado que a taxa de 5xx do `donation-service` passa de 5% em 5 minutos
 Quando o Monitor do Datadog entra em Alert
 Então um incidente é aberto no PagerDuty e o Discord é notificado
 E o `workflow self-heal.yaml` executa `rollout restart` em `donation/donation-service`

Cenário: Restauração após perda do namespace
 Dado um backup Velero do namespace "donation"
 Quando o namespace é removido e o `restore` é executado
 Então os recursos do namespace voltam
 E um novo `POST /donations` retorna 201

SEÇÃO 9

Requisitos Técnicos do Enunciado (Frentes 0 a 4)

Resumo: 15 requisitos, dos quais 5 implementados, 9 implementados com ressalva e 1 não evidenciado. Nenhuma evidência visual consta na pasta.

A coluna "Critério de aceite" é proposta desta especificação para a demonstração; as colunas "Implementação" e "Status" descrevem o código.

9.1 Frente 0 — Fundação DevOps

ID	Requisito [ENUNCIADO]	Implementação	Critério de aceite [ANÁLISE]	Status	Evidência
REQ-F0.1	Dockerfiles otimizados + Kubernetes gerenciado	Multi-stage nos 4 repositórios (Go estático <code>-s -w</code> ; Python slim); AKS	Imagens buildadas e pods <code>Running</code>	Ressalva sem <code>.dockerignore</code> ; sem instrução <code>USER</code> nem <code>securityContext</code>	Pendente
REQ-F0.2	Terraform para cluster, bancos, mensageria e rede	14 arquivos <code>.tf</code> , state remoto	<code>terraform plan</code> sem drift com <code>deploy_databases = true</code>	Implementado	Pendente
REQ-F0.3	CI/CD com testes, SAST/SCA e build	<code>build-push.yaml</code> , 5 jobs	Execução verde na <code>main</code>	Ressalva SonarQube não bloqueia; <code>trivy-action@master</code> (RSK-11)	Pendente
REQ-F0.4	GitOps (ArgoCD/FluxCD)	App-of-Apps + ApplicationSet + addons	Applications Synced/Healthy	Ressalva o próprio código orienta <code>kubectl apply</code> para o bootstrap e para os Secrets (README; comentário do <code>apps-appset.yaml</code>); ver RSK-05 a RSK-07	Pendente
REQ-F0.5	Prometheus/Grafana/Loki/OTel + APM com Distributed Tracing	Stack completa + Datadog; trace context na fila	Trace <code>donation</code> → <code>notification</code> visível	Implementado	Pendente

9.2 Frente 1 — SRE

ID	Requisito [ENUNCIADO]	Implementação	Critério de aceite [ANÁLISE]	Status	Evidência
REQ-F1.1	≥ 2 SLIs + SLO cada, no <code>donation-service</code>	Sucesso ≥ 99,9% (30 d); p95 ≤ 500 ms (5 min)	PromQL retorna valor	Ressalva SLIs incluem <code>/health</code> (RSK-08)	Texto
REQ-F1.2	Dashboard exclusivo de SLOs e Error Budget	<code>solidarytech-sre-dashboard</code>	Painel com dados	Implementado	Pendente

ID	Requisito [ENUNCIADO]	Implementação	Critério de aceite [ANÁLISE]	Status	Evidência
REQ-F1.3	Mostrar como a stack reduz o MTTR	Cadeia automática em código (<code>datadog.tf</code> , <code>pagerduty.tf</code> , <code>self-heal.yml</code>)	Timestamps medidos neste ambiente	Implementado	Pendente sem medição na pasta (RSK-04)

9.3 Frente 2 — FinOps

ID	Requisito [ENUNCIADO]	Implementação	Critério de aceite [ANÁLISE]	Status	Evidência
REQ-F2.1	Tags <code>Project</code> , <code>Environment</code> e <code>CostCenter</code> no Terraform	<code>local.mandatory_tags</code> em 9 recursos	Tag Editor / Cost Management filtrado	Implementado	Pendente
REQ-F2.2	Rightsizing de requests/limits via GitOps com base em métricas de CPU/memória	Requests/limits definidos; painel de CPU	Commit de ajuste com métrica antes e depois	Ressalva painel só de CPU; sem registro de ajuste	Pendente
REQ-F2.3	Forecast mensal + ≥ 1 recomendação nativa	Forecast no documento 02; Cosmos Serverless no código; Reserved Instances recomendado no documento 02 §4	Tabela com premissas	Ressalva itens do código fora do forecast (RSK-15)	Texto

9.4 Frente 3 — ITSM e AIOps

ID	Requisito [ENUNCIADO]	Implementação	Critério de aceite [ANÁLISE]	Status	Evidência
REQ-F3.1	Ativar a IA do APM	Sem configuração em código (<code>datadog.tf</code>)	Watchdog Insight visível	Não evidenciado	Pendente
REQ-F3.2	Ciclo de vida do incidente, da detecção ao post-mortem e à comunicação	Cadeia automática em código (§12.6); processo completo no documento 04	Diagrama + incidente real	Ressalva a Sev2 do documento 04 §2 não tem Monitor com o <code>monitored_services</code> padrão (RSK-09)	Texto

9.5 Frente 4 — Multicloud, Segurança e DR

ID	Requisito [ENUNCIADO]	Implementação	Critério de aceite [ANÁLISE]	Status	Evidência
REQ-F4.1	PCN com RTO e RPO para os dados de doações	Metas de RPO/RTO no código (<code>terraform/README.md</code>); PCN no documento 03	RPO/RTO + restore cronometrado	Ressalva perda de região fora da cobertura (RSK-02); RTO não medido	Texto

ID	Requisito [ENUNCIADO]	Implementação	Critério de aceite [ANÁLISE]	Status	Evidência
REQ-F4.2	DR prático: Opção A (Velero para bucket externo) ou Opção B (espelho em outra região)	Opção A: <code>velero.tf</code> + <code>addons/velero/</code>	Backup, schedules e restore em vídeo	Ressalva backups na mesma região, storage LRS	Pendente

SEÇÃO 10

SRE — Confiabilidade e Golden Metrics

10.1 Golden Metrics

Golden Metric	Métrica no código	Observação
Latência	<code>solidarytech_http_request_duration_seconds</code>	Histograma de 5 ms a 10 s
Tráfego	<code>solidarytech_http_requests_total</code>	Contador por método, rota e status
Erros	<code>solidarytech_http_requests_total{http_status_code=~"5.."} </code>	—
Saturação	<code>container_cpu_usage_seconds_total / kube_pod_container_resource_requests</code>	Painel Rightsizing, só CPU

10.2 SLI, SLO e SLA

Nível	Definição	Meta	Fonte
SLI 1	% de requisições não-5xx em 30 d (<code>or vector(0)</code> para ausência de 5xx)	—	Dashboard SRE
SLI 2	p95 da latência em 5 min	—	Dashboard SRE
SLO 1	Taxa de sucesso	≥ 99,9% em 30 d	Dashboard SRE (texto do painel)
SLO 2	Latência p95	≤ 500 ms	Dashboard SRE; <code>latency_p95_threshold_ms = 500</code>
SLA	Disponibilidade mensal do fluxo de doações	≥ 99,5%	Definido no documento 01 §5 (compromisso de negócio, não é configuração de código)

Error Budget: o gauge do dashboard divide a taxa de erro de 30 d por `0.001` , que é o orçamento de 0,1% do SLO 1. Em tempo, 0,1% de 30 dias equivale a 43,2 min ($0,001 \times 30 \times 24 \times 60$).

Os SLIs filtram só `service_name="donation-service"` , então incluem as chamadas de `/health` das probes (readiness a cada 10 s, liveness a cada 15 s) (RSK-08).

10.3 Dashboards

SolidaryTech - SRE (SLO & Error Budget) (`uid: solidarytech-sre` , refresh de 1 min):

Painel	Tipo	Limiares
SLO de Disponibilidade / SLO de Latência	texto	—
Disponibilidade Real (30d)	stat	vermelho < 99,9 · amarelo 99,9–99,95 · verde ≥ 99,95
Error Budget Restante (30d)	gauge	vermelho < 20 · amarelo 20–50 · verde ≥ 50
Total de Requisições (30d) · Total de Erros 5xx (30d)	stat	5xx ≥ 1 em vermelho
Taxa de sucesso (janela 1h) vs. SLO (99.9%)	série	linha em 99,9

Painel	Tipo	Limiares
Latência p95 (ao vivo, 5m) · Latência p95 ao longo do tempo vs. SLO (500ms)	stat + série	vermelho a partir de 0,5 s

SolidaryTech - Overview (uid: solidarytech-overview , refresh de 30 s): CPU e memória por nó; restarts de pods na última 1 h; Rightsizing (CPU usada vs. request, agrupado por namespace); taxa de requisições e de erros 5xx por serviço; logs dos microsserviços via Loki.

10.4 Consultas de Referência

```
# SLI 1 (dashboard SRE)
100 * (1 - ((sum(increase(solidarytech_http_requests_total{service_name="donation-service",
http_status_code=~"5.."}[30d])) or vector(0))
/ sum(increase(solidarytech_http_requests_total{service_name="donation-service"}[30d]))))

# SLI 2 (dashboard SRE)
histogram_quantile(0.95, sum by (le)
(rate(solidarytech_http_request_duration_seconds_bucket{service_name="donation-service"}[5m])))

# Rightsizing (dashboard Overview)
100 * sum by (namespace)
(rate(container_cpu_usage_seconds_total{namespace=~"ngo|donation|volunteer|notification"}[5m]))
/ sum by (namespace)
(kube_pod_container_resource_requests{namespace=~"ngo|donation|volunteer|notification", resource="cpu"})
```

PROMQL

```
{k8s_namespace_name=~"ngo|donation|volunteer|notification"}
```

LOGQL

10.5 Política de Error Budget

A política de uso do Error Budget está definida no documento 01 §4.

[ANÁLISE] Proposta, não formalizada: usar como faixas de decisão os mesmos limiares já configurados no gauge do dashboard.

Budget restante	Cor no gauge	Ação proposta
≥ 50%	verde	Operação normal
20–50%	amarelo	Aviso à squad no Discord
< 20%	vermelho	Congelar deploys não críticos

10.6 Cadeia de Resposta Configurada (MTTR)

Etapa	Configuração	Fonte
Detecção	Monitor 5xx > 5% em 5 min; Monitor p95 > 500 ms em 5 min	datadog.tf
Incidente	@pagerduty-<svc> → Service no PagerDuty, escalonamento em 10 min	datadog.tf ; pagerduty.tf
Notificação	@webhook-discord-alerts	datadog.tf
Mitigação	@webhook-github-selfheal-<svc> (só no Monitor de 5xx) → repository_dispatch → kubectl rollout restart + rollout status --timeout=120s	datadog.tf ; self-heal.yml

Etapa	Configuração	Fonte
Evidência	<code>kubectl get pods -n <ns> -o wide</code> com <code>if: always()</code>	<code>self-heal.yml</code>

Medição de MTTR: não há na pasta (RSK-04, EVD-10).

SEÇÃO 11

FinOps — Tagueamento, Custos e Rightsizing

11.1 Estratégia de Tagging

```
# fiap-tc-5-terraform/main.tf (trecho)
locals {
  mandatory_tags = {
    Project      = "SolidaryTech"
    Environment  = "Production"
    CostCenter   = "NGO-Core"
  }

  tags = merge(local.mandatory_tags, {
    ManagedBy = "terraform"
    Fase      = "5"
  })
}
```

HCL

Recursos com tags (9):

Recurso	Component
Resource Group	core
VNet	network
ACR	acr
AKS	aks
PostgreSQL ngo	ngo-service
PostgreSQL donation	donation-service (+ Criticality = "hot-path")
Service Bus namespace	messaging
Cosmos DB account	nosql
Storage do Velero	disaster-recovery

Sem tags no código: subnet, bancos, regras de firewall, fila, tabelas, container, role assignments e node pool do AKS.

11.2 Recursos e Quantidades que Compõem o Custo

Recurso	Quantidade / configuração no código	Fonte
AKS	Nós Standard_B2s , mínimo 3, máximo 5	variables.tf ; aks.tf
ACR	1 registry Basic	aks.tf
PostgreSQL Flexible	2 servidores B_Standard_B1ms , 32 GB cada	postgresql.tf ; variables.tf
Service Bus	1 namespace Basic + 1 fila	servicebus.tf ; variables.tf
Cosmos DB	1 conta Serverless, 2 tabelas	cosmosdb.tf

Recurso	Quantidade / configuração no código	Fonte
Storage do Velero	1 conta Standard LRS	<code>velero.tf</code>
Services LoadBalancer	2: <code>ingress-nginx</code> e ArgoCD	<code>argocd.tf</code>
Volumes (PVC)	Prometheus 20 Gi, Loki 10 Gi, Grafana 2 Gi, Alertmanager 1 Gi, SonarQube 5 Gi + 5 Gi	values dos addons

Os valores monetários (preço unitário e total mensal) estão no documento 02 §2. Segundo o próprio documento, os preços unitários vêm da Azure Retail Prices API (exceto o Load Balancer, por valor de lista) e os volumes de uso são estimativas — fontes externas ao código.

11.3 Otimizações

#	Otimização	Situação no código	Registro	Fonte
1	Cosmos DB Serverless em vez de throughput provisionado	Aplicada desde o commit <code>82421a8</code>	Comentário do <code>cosmosdb.tf</code> : 400 RU/s por tabela (800 RU/s no total) custariam ~US\$ 46,72/mês fixos; o serverless cobra US\$ 0,25 por milhão de RUs	<code>cosmosdb.tf</code> ; [GIT]
2	Reserved Instances / Savings Plan para os nós	Não aplicada: nenhum recurso de reserva no Terraform	Recomendação no documento 02 §4	<code>*.tf</code>

11.4 Rightsizing

Workload	Requests	Limits	Escala
<code>donation-service</code>	100m / 64Mi	500m / 256Mi	HPA 2–10 @ CPU 70%
<code>ngo-service</code>	50m / 64Mi	200m / 128Mi	2 réplicas
<code>volunteer-service</code>	50m / 96Mi	250m / 192Mi	2 réplicas
<code>notification-service</code>	50m / 64Mi	200m / 128Mi	2 réplicas

O comentário do manifesto do `ngo-service` classifica seus valores como iniciais conservadores, a revisar com o painel "Rightsizing" do dashboard Overview após alguns dias de tráfego real. O painel compara só CPU.

11.5 Itens do Código Fora do Forecast

A tabela do documento 02 §2 não soma:

- o Service do ArgoCD do tipo `LoadBalancer` (`argocd.tf`);
- os PVCs do SonarQube (5 Gi + 5 Gi) e do Alertmanager (1 Gi);
- os serviços SaaS (Datadog, PagerDuty).

O documento 02 §2 registra esses itens em nota abaixo da tabela.

SEÇÃO 12

ITSM e AIOps — Gestão de Incidentes

12.1 Monitors

Monitor	Query	Limiar	Notifica	Renotificação
[SolidaryTech] Taxa de erros 5xx alta - <svc> (BR-01)	trace.http.server.request.errors / .hits × 100 , sum(last_5m)	> 5	@pagerduty-<svc> @webhook-discord-alerts @webhook-github-selfheal-<svc>	10 min
[SolidaryTech] Latência p95 alta - <svc> (BR-02)	p95:trace.http.server.request, avg(last_5m)	> 500	@pagerduty-<svc> @webhook-discord-alerts	—

Os dois usam `notify_no_data = false`, `require_full_window = false` e as tags `service:<svc>`, `env:production`, `team:solidarytech` e `fase:5` (o de latência também `slo:latency`). A mensagem de recuperação avisa quando a métrica normaliza. A cobertura segue `var.monitored_services`, cujo padrão é `["donation-service"]`.

12.2 PagerDuty

Há um `datadog_integration_pagerduty_service_object` por serviço monitorado. A Escalation Policy "SolidaryTech - On-call" escala em 10 min, repete 2 loops e tem como alvo 1 usuário (`pagerduty.tf`).

12.3 ChatOps

O webhook `discord-alerts` publica `$ALERT_TITLE`, `$EVENT_MSG`, `$ALERT_TRANSITION`, `$ALERT_PRIORITY` e `$LINK` (`datadog.tf`).

12.4 Self-Healing

Aspecto	Implementação (<code>self-heal.yml</code>)
Gatilhos	<code>repository_dispatch</code> (<code>types: [self-heal]</code>) ou <code>workflow_dispatch</code>
Escopo	<code>SERVICE_NAME</code> e <code>NAMESPACE</code> fixos: cada repositório reinicia só o próprio serviço; o webhook escolhe o repositório por <code>var.service_repos</code>
Permissões	<code>permissions: contents: read</code> ; <code>azure/login@v2</code> com <code>AZURE_CREDENTIALS</code>
Ação	<code>kubectl rollout restart + rollout status --timeout=120s</code>
Evidência	Passo com <code>if: always()</code> : <code>kubectl get pods -n <ns> -o wide</code>

12.5 AIOps — Watchdog

O código não configura o Watchdog: não existe resource do provider Terraform para ele, e o comentário do `datadog.tf` registra que ele é automático assim que há dados de APM ou de infraestrutura chegando à conta. O mesmo comentário diz que a evidência (Watchdog Insights detectando anomalia) seria mostrada ao vivo no vídeo, que não consta na pasta.

Status: não evidenciado.

12.6 Cadeia Automática de Incidente

Etapas configuradas em código (`datadog.tf`, `pagerduty.tf`, `self-heal.yml`):

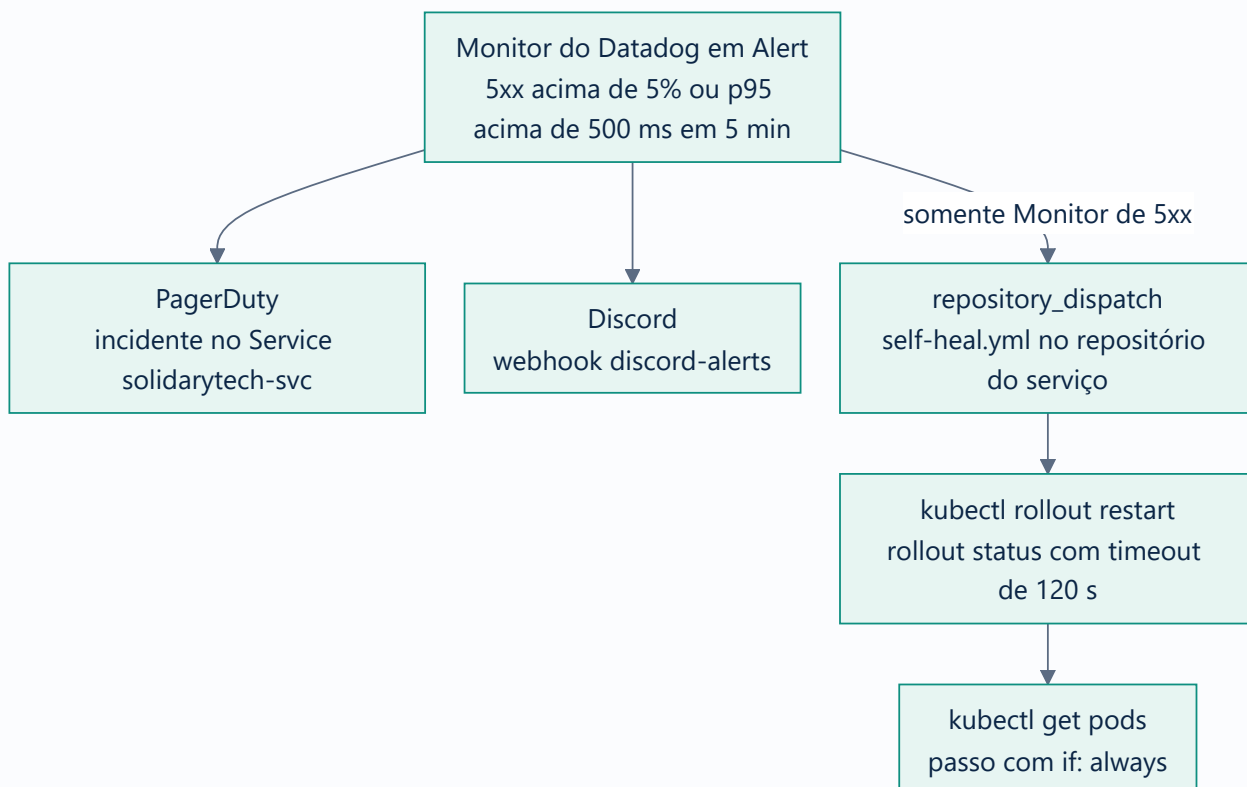


Figura 2 — Cadeia Automática de Incidente

O processo completo de gestão do incidente — severidades, confirmação, post-mortem e comunicação aos stakeholders — está no documento 04. Esses prazos e papéis são definições de processo; não há configuração de código para eles além do escalonamento do PagerDuty (§12.2).

12.7 Alertas Acionáveis × Painéis de Diagnóstico

Só os dois Monitors do Datadog acionam PagerDuty e Discord, e só o de erro 5xx aciona o self-heal (`datadog.tf`). Os painéis do dashboard Overview não têm regras de alerta no repositório. O comentário de `kube-prometheus-stack/values.yaml` registra que o Alertmanager fica ativo para alertas de infraestrutura do Prometheus.

SEÇÃO 13

Segurança, PCN e Disaster Recovery

13.1 RPO e RTO

Serviço	RPO	RTO	Onde está registrado
donation-service	≤ 5 min (PITR)	≤ 30 min	terraform/README.md ; comentários de postgresql.tf e schedule.yaml
ngo, volunteer, notification	≤ 15 min	≤ 2 h	terraform/README.md

Limite da configuração: os backups ficam na mesma região. O storage do Velero é LRS em `eastus`, o PostgreSQL não tem backup geo-redundante e o Cosmos DB tem 1 região. A perda da região não está coberta; o documento 03 §1 declara esse limite.

13.2 Mecanismos

Dado	Mecanismo	Configuração
donation_db	PITR (retenção de 14 d)	backup_retention_days = 14
ngo_db	PITR (retenção de 7 d)	backup_retention_days = 7
Cosmos DB	Sem configuração de backup no código	cosmosdb.tf não declara bloco backup (o documento 03 §3.1 registra que vale o padrão da plataforma)
Estado do cluster	Velero	Schedules abaixo

Schedule	Escopo	Cron	TTL
solidarytech-donation-hourly	donation	0 * * * *	168 h
solidarytech-daily	ngo, donation, volunteer, notification	0 2 * * *	720 h

Os dois Schedules têm `includeClusterResources: true` e `snapshotVolumes: true`. O SP do Velero tem `Storage Account Contributor`, porque o plugin chama `listKeys`, e `Contributor` no node resource group (`velero.tf`).

13.3 Evidência de Execução

A pasta **não contém** log, print ou vídeo de backup ou restore (EVD-08). O histórico Git registra duas correções no Velero: `5b4fa3b` (placeholders do `values.yaml` substituídos) e `3d5258e` (role `Storage Account Contributor`) [\[GIT\]](#). O documento 03 §4 traz o relato textual de uma execução.

13.4 Runbook e Papéis

O procedimento de restore está no documento 03 §5, e os papéis durante o DR no documento 03 §6. Os nomes usados no runbook conferem com o código: os Schedules `solidarytech-donation-hourly` e `solidarytech-daily`, o namespace `donation`, a Application `app-donation` (gerada pelo ApplicationSet) e o servidor `pg-donation-<prefix>`.

13.5 Opção A × Opção B

Está implementada a Opção A (Velero). O Terraform declara um único cluster AKS, sem ambiente espelho em outra região. A justificativa da escolha está no documento 03 §7.

13.6 Controles de Segurança Implementados

Controle	Implementação	Fonte
Shift-left	Trivy FS + imagem (CRITICAL com correção bloqueia); SonarQube; lint	<code>build-push.yaml</code>
Imagem imutável	Tag por SHA nos overlays	<code>kustomization.yaml</code>
Criptografia em trânsito	<code>sslmode=require</code> ; TLS 1.2 no storage; Cosmos HTTPS	<code>outputs.tf</code> ; <code>velero.tf</code>
Backup privado	<code>container_access_type = "private"</code>	<code>velero.tf</code>
Identidades dedicadas	SPs do Veleró e do self-heal criados pelo Terraform	<code>velero.tf</code> ; <code>github_actions.tf</code>
Permissão mínima no workflow	<code>permissions: contents: read</code>	<code>self-heal.yml</code>

As lacunas estão em §19.

SEÇÃO 14

Estrutura dos Repositórios

TEXTO

```
fiap-tc-5-terraform/
├── README.md · .gitignore · .terraform.lock.hcl · terraform.tfvars.example
├── main.tf · providers.tf · variables.tf · outputs.tf
├── network.tf · aks.tf · argocd.tf
├── postgresql.tf · servicebus.tf · cosmosdb.tf · velero.tf
└── github_actions.tf · datadog.tf · pagerduty.tf

fiap-tc-5-gitops/
├── README.md
├── bootstrap/app-of-apps.yaml
├── applicationsets/{addons.yaml, apps-appset.yaml}
├── addons/{kustomization.yaml, kube-prometheus-stack/, loki/, otel-collector-gateway/,
    otel-collector-logs/, grafana-dashboards/, sonarqube/, velero/}
├── apps/{ngo,donation,volunteer,notification}/
│   ├── base/{deployment.yaml, service.yaml, kustomization.yaml}
│   └── overlays/prod/{kustomization.yaml, secrets.yaml, ...}
│       # ngo, donation: configmap.yaml + db-init-job.yaml + ingress.yaml · donation: hpa.yaml · volunteer: ingress.yaml
└──

fiap-tc-5-donation-service/  main.go handlers.go handlers_test.go otel.go queue.go servicebus.go sqs.go types.go
                             db/init.sql Dockerfile go.mod go.sum README.md .github/workflows/{build-push.yaml,self-heal.yml}

fiap-tc-5-ngo-service/      app.py test_app.py requirements.txt Dockerfile README.md db/init.sql .github/workflows/...

fiap-tc-5-volunteer-service/ app.py test_app.py requirements.txt Dockerfile README.md .github/workflows/...

fiap-tc-5-notification-service/ app.py test_app.py requirements.txt Dockerfile README.md .github/workflows/...
```

SEÇÃO 15

Fluxos End-to-End

15.1 Deploy

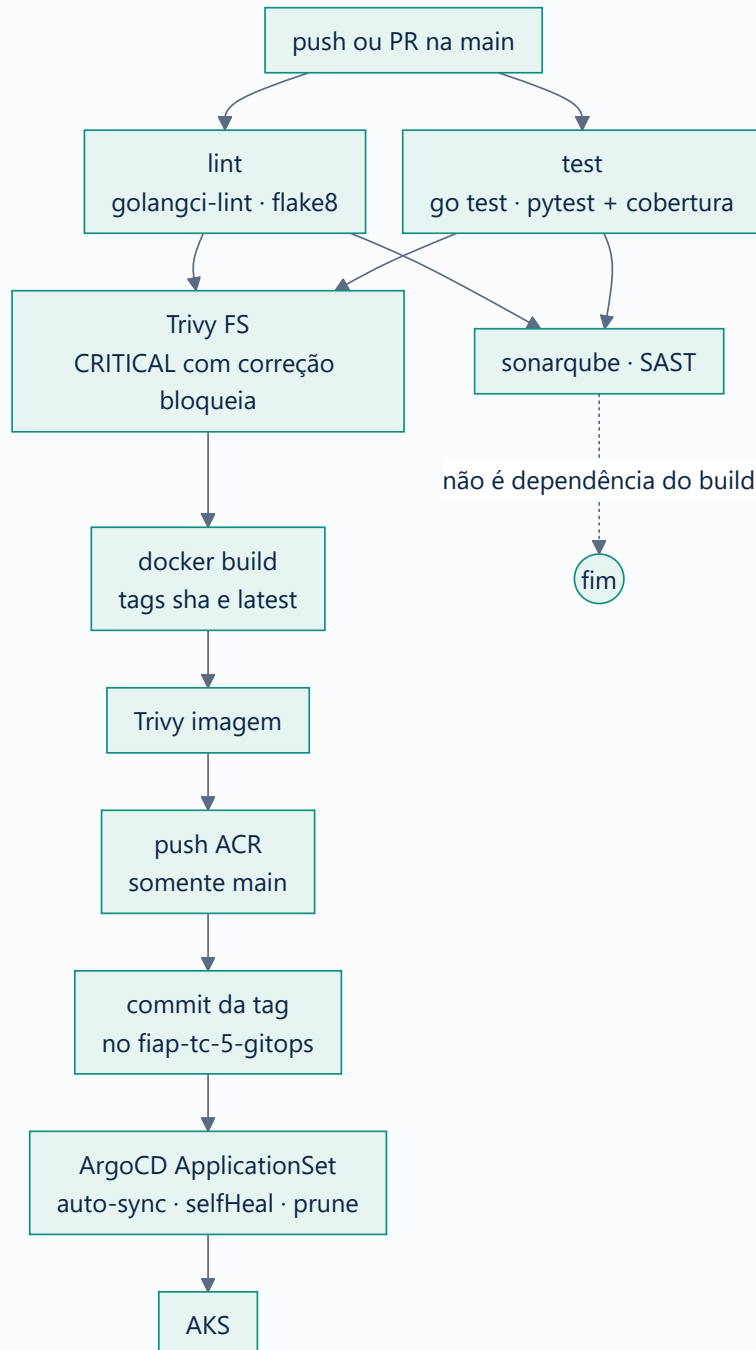


Figura 3 — Deploy

1. Push ou PR na `main` dispara o `build-push.yaml`.
2. `lint` e `test` rodam em paralelo; a cobertura vira artefato.
3. `sonarqube` e `build-scan-push` dependem de `[lint, test]` e rodam em paralelo.

4. `build-scan-push`: Trivy FS → `docker build` → Trivy imagem → login e push no ACR (só `main`, fora de PR).
5. `update-gitops`: `kustomize edit set image` e commit `deploy: update <svc> to <sha>`.
6. O ArgoCD sincroniza `app-<svc>` (`auto-sync`, `selfHeal`, `prune`).
7. O Deployment define readiness probe em `/health` a cada 10 s e liveness probe a cada 15 s.

15.2 Doação

1. `POST /donations` pelo Ingress chega ao `donation-service:8082`.
2. `otelhttp` abre o span e `withMetrics` registra as métricas.
3. INSERT com `status = APPROVED` → 201.
4. Uma goroutine publica o evento em `donation-events` com `traceparent`.
5. O worker do `notification-service` consome, abre o span CONSUMER e grava em `DonationNotifications`.

15.3 Incidente

1. A taxa de 5xx passa de 5% em 5 min (BR-01).
2. O Monitor dispara PagerDuty, Discord e `repository_dispatch`.
3. `self-heal.yml` faz `rollout restart` + `rollout status` e lista os pods.
4. Quando a métrica normaliza, o Monitor envia a mensagem de recuperação configurada no `datadog.tf`.

SEÇÃO 16

Estratégia de Validação

16.1 Roteiro de Verificação por Camada [ANÁLISE]

Os comandos usam os nomes reais do código.

Camada	Como validar
Infraestrutura	<code>terraform plan</code> sem drift; recursos com tags no portal
GitOps	<code>kubectl get applications -n argocd</code> : <code>app-*</code> e <code>addon-*</code> em Synced/Healthy
Aplicações	<code>kubectl get pods -n donation</code> ; <code>curl http://<ip-ingress>/donations</code>
OTel Collector	<code>kubectl logs -n monitoring deploy/otel-collector</code>
Métricas / Logs	<code>solidarytech_http_requests_total</code> no Grafana; LogQL da §10.4
Traces	Datadog APM / Service Map: <code>donation-service</code> → <code>notification-service</code>
Alertas	Monitors [SolidaryTech] ... no Datadog; Service <code>solidarytech-donation-service</code> no PagerDuty
Self-healing	<code>gh workflow run self-heal.yml --repo freitasleoalves/fiap-tc-5-donation-service</code>
DR	<code>velero backup get</code> ; <code>velero schedule get</code> ; restore do <code>donation</code> cronometrado

16.2 Testes Automatizados Existentes

Serviço	Framework	Testes	Cobrem
<code>donation-service</code>	<code>go test</code>	3	health; 405; 400 com payload inválido
<code>ngo-service</code>	<code>pytest</code>	4	health; 400; 201 com pool mockado; listagem
<code>volunteer-service</code>	<code>pytest</code>	4	health; 400; 201; listagem
<code>notification-service</code>	<code>pytest</code>	3	health; mensagem válida; JSON inválido
Total		14	A cobertura vai para o SonarQube

Não há testes de integração nem de carga na pasta.

16.3 Comandos de Referência (outputs do Terraform)

```
az aks get-credentials --resource-group rg-solidarytech-prod --name aks-solidarytech-prod
kubectl get svc ingress-nginx-controller -n ingress-nginx -o
jsonpath='{.status.loadBalancer.ingress[0].ip}'
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}' | base64 -d
```

BASH

SEÇÃO 17

Entregáveis e Checklist do Enunciado

17.1 Checklist [ENUNCIADO]

Requisito	Evidência no projeto	Situação
Dockerfiles otimizados + Kubernetes	4 <code>Dockerfile</code> multi-stage; <code>aks.tf</code>	Ressalva
Terraform: cluster, bancos, mensageria, rede	<code>fiap-tc-5-terraform</code>	OK
CI/CD com testes, SAST/SCA, build	<code>build-push.yaml</code> ×4	Ressalva
GitOps	<code>fiap-tc-5-gitops</code> + <code>argocd.tf</code>	Ressalva
Prometheus, Grafana, Loki, OTel	<code>addons/</code>	OK
APM com Distributed Tracing	Exporter Datadog + trace context na fila	OK
≥ 2 SLIs + SLO	Dashboard SRE; 01	Ressalva
Dashboard SRE	<code>solidarytech-sre-dashboard.yaml</code>	OK
MTTR no relatório	Cadeia configurada em código; sem medição na pasta	Pendente
Tags obrigatórias	<code>main.tf</code>	OK
Rightsizing	<code>deployment.yaml</code> + painel	Ressalva
Forecast + recomendação	02	Ressalva
AIOps	—	Não evidenciado
Ciclo de vida do incidente	§12.6; 04	Ressalva
PCN com RPO/RTO	<code>terraform/README.md</code> ; 03	Ressalva
DR prático	Opção A	Ressalva

17.2 Artefatos

ID	Artefato	Localização	Status
DEL-01	IaC com tags	<code>fiap-tc-5-terraform</code>	Pronto
DEL-02	YAML com limits/requests	<code>fiap-tc-5-gitops</code>	Pronto
DEL-03	Pipelines DevSecOps	<code>build-push.yaml</code> ×4	Pronto
DEL-04	Vídeo (≤ 20 min): pitch + demo	Roteiro do pitch em 05 ; vídeo não consta	Pendente
DEL-05	PDF: nomes, RMs e usernames	Capa e §1.1	Pronto
DEL-06	PDF: links do repositório e do vídeo	Repositórios em §22.2; vídeo não consta	Parcial

ID	Artefato	Localização	Status
DEL-07 a DEL-10	PDF: seções SRE, FinOps, Segurança/DR e ITSM	01 a 04	Texto pronto; prints pendentes

17.3 Plano de Evidências

A regra de avaliação do enunciado (p. 6) prevê dedução direta de pontos para requisito que não for claramente demonstrado no vídeo ou documentado no relatório. **A pasta não contém imagens nem vídeo.**

ID	Evidência	Requisitos
EVD-01	Pipeline verde, do lint ao update-gitops	REQ-F0.1, F0.3
EVD-02	Applications Synced/Healthy + sync disparado por commit	REQ-F0.4
EVD-03	Recursos com as tags obrigatórias	REQ-F2.1, F0.2
EVD-04	Trace donation-service → notification-service	REQ-F0.5
EVD-05	Monitors + incidente no PagerDuty + mensagem no Discord	REQ-F3.2
EVD-06	Dashboard SRE com dados	REQ-F1.1, F1.2
EVD-07	Watchdog Insight	REQ-F3.1
EVD-08	Backup, schedules e restore cronometrado	REQ-F4.1, F4.2
EVD-09	Painel de rightsizing + commit de ajuste	REQ-F2.2
EVD-10	Incidente controlado com timestamps	REQ-F1.3

SEÇÃO 18

Matriz de Rastreabilidade

Negócio	Requisitos	Artefatos	Evidência	Situação
RN-01	REQ-F4.1, F4.2 · RF-18, RF-19 · RNF-05, RNF-06	velero.tf ; addons/velero/ ; postgresql.tf ; 03	EVD-08	RSK-02
RN-02	REQ-F2.1–F2.3 · RF-10 · RNF-13	main.tf ; deployment.yaml ; 02	EVD-03, EVD-09	Evidência pendente
RN-03	REQ-F3.1 · RF-17	datadog.tf ; 04	EVD-07	Não evidenciado
RN-04	REQ-F1.1, F1.2 · RF-14 · RNF-01–RNF-03	otel.go ; dashboard SRE; 01	EVD-06	RSK-08
RN-05	RNF-04	hpa.yaml ; aks.tf	—	RSK-05
RN-06	REQ-F1.x · RNF-05, RNF-06	postgresql.tf ; schedule.yaml	EVD-06, EVD-08	OK
RN-07	DEL-04	05	Vídeo	Pendente
RN-08	REQ-F0.1–F0.5 · RF-11–RF-13 · RNF-10, RNF-11	Pipelines; bootstrap/ ; applicationsets/	EVD-01, EVD-02	RSK-06, RSK-07
D3 (MTTR)	REQ-F1.3 · RF-15, RF-16 · RNF-08, RNF-09	datadog.tf ; pagerduty.tf ; self-heal.yml	EVD-05, EVD-10	Pendente (RSK-04)

SEÇÃO 19

Riscos

19.1 Credenciais Reais Versionadas (RSK-01 · Crítico)

Fatos:

- Os 6 arquivos `secrets.yaml` do `fiap-tc-5-gitops` estão com valores reais: nenhum placeholder `REPLACE_WITH_` restante.
- Os valores entraram no commit `b7b9ad7` ("preenche secrets reais") `[GIT]`, e a chave do Datadog foi atualizada depois em `9ce7b3e` `[GIT]`.
- O repositório é declarado público em `argocd.tf` e nos cabeçalhos dos próprios arquivos.

Estão expostos:

- credenciais do SP do Velero, que tem `Contributor` no node resource group;
- a API key do Datadog;
- as connection strings de PostgreSQL, Service Bus e Cosmos DB.

Também no código: as senhas de admin do Grafana e do SonarQube estão em texto nos respectivos `values.yaml` (`kube-prometheus-stack/` e `sonarqube/`). O `README.md` do GitOps e comentários das Applications ainda afirmam que os arquivos só têm placeholders ([Anexo B.2](#)).

Não há Sealed Secrets nem External Secrets no repositório.

[ANÁLISE] Rotacionar todas as credenciais; tornar o repositório privado ou remover os valores do histórico; adotar um mecanismo de segredos fora do Git.

19.2 Exposição de Dados Pessoais (RSK-03 · Alto)

Fatos: as APIs não têm autenticação (LIM-01). Os Ingress não têm bloco `tls`. O ArgoCD tem Service `LoadBalancer` próprio. O SonarQube responde no path `/`.

[ANÁLISE] TLS nos Ingress; ArgoCD sem exposição direta; autenticação nas leituras; restringir o SonarQube.

19.3 DR Sem Cobertura Regional (RSK-02 · Alto)

Fatos: storage do Velero LRS em `eastus`; PostgreSQL com `geo_redundant_backup_enabled = false` e sem bloco `high_availability`; Cosmos DB com 1 `geo_location`. O enunciado chama a Opção A de *Multicloud/Cross-Region Backup*. O documento 03 §1 declara esse limite.

[ANÁLISE] Replicar os backups para outra região ou nuvem, ou manter o limite declarado no PCN.

19.4 Demais Achados

ID	Sev.	Fato verificado	[ANÁLISE] Recomendação
RSK-04	Alto	A pasta não contém medição de MTTR, evidência do Watchdog, prints nem vídeo	Incidente controlado com timestamps (EVD-05, 07, 10)
RSK-05	Médio	O Deployment do <code>donation</code> declara <code>replicas: 2</code> , o HPA vai de 2 a 10 e o ApplicationSet usa <code>selfHeal</code> com <code>ignoreDifferences</code> só para Secrets	Verificar em execução se o ArgoCD marca <code>OutOfSync</code> quando o HPA escala; se marcar, tirar <code>replicas</code> do Git ou ignorar o campo

ID	Sev.	Fato verificado	[ANÁLISE] Recomendação
RSK-06	Médio	O commit <code>9ce7b3e</code> registra que o <code>ignoreDifferences</code> "não está blindando o <code>selfHeal</code> ". Nenhuma Application declara a <code>syncOption RespectIgnoreDifferences=true</code> (as <code>syncOptions</code> presentes são <code>CreateNamespace=true</code> e <code>ServerSideApply=true</code>)	Avaliar essa <code>syncOption</code> junto com a solução de segredos (RSK-01)
RSK-07	Médio	O Application do Velero usa <code>directory.include:</code> " <code>secrets.yaml,schedule.yaml</code> ", e o comentário do arquivo afirma que o <code>include</code> aceita lista separada por vírgula. O documento 03 §4 relata que o <code>schedule.yaml</code> não foi aplicado pelo ArgoCD	Confirmar em execução se o ArgoCD aplica o <code>schedule.yaml</code> e, se não aplicar, corrigir a sintaxe do <code>include</code>
RSK-08	Médio	Os SLIs medem todas as rotas, inclusive <code>/health</code>	Filtrar <code>http_route="/donations"</code>
RSK-09	Médio	O Monitor de erro dispara em 5% em 5 min, 50× o orçamento do SLO (0,1% em 30 d). Nenhum Monitor acompanha o consumo do Error Budget. A Sev2 do documento 04 §2 (ngo, volunteer, notification) não tem Monitor com o <code>monitored_services</code> padrão	Alerta sobre consumo do Error Budget; ampliar <code>monitored_services</code> (Q-04)
RSK-10	Médio	Perda silenciosa de eventos (LIM-04, LIM-05)	Retry no envio; abandonar ou mandar para dead-letter em caso de falha
RSK-11	Médio	<code>build-scan-push</code> depende só de <code>[lint, test]</code> , então o SonarQube não bloqueia; sem <code>sonar.qualitygate.wait</code> ; <code>aquasecurity/trivy-action@master</code> referencia uma branch; <code>ignore-unfixed: "true"</code> deixa passar CRITICAL sem correção	Tornar o SonarQube um gate; fixar a versão da action; registrar a política de CVE sem correção
RSK-12	Médio	<code>AZURE_CREDENTIALS</code> com <code>client secret (outputs.tf)</code> ; ACR com <code>admin_enabled = true</code> e <code>outputs</code> de usuário e senha <code>admin (aks.tf , outputs.tf)</code> , com o CI fazendo login por usuário e senha (<code>ACR_USERNAME / ACR_PASSWORD</code>); SP de self-heal com <code>Azure Kubernetes Service Cluster Admin Role</code> , que o comentário do <code>github_actions.tf</code> justifica para o <code>rollout restart</code>	Credencial sem segredo de longa duração; papel restrito ao restart
RSK-13	Médio	PostgreSQL com <code>public_network_access_enabled = true</code> e regra <code>AllowAzureServices (0.0.0.0)</code> ; sem private endpoint ou integração com a VNet no código	Acesso privado ao banco
RSK-14	Baixo	Sem <code>USER</code> nos Dockerfiles; sem <code>securityContext</code> ; sem <code>.dockerignore</code> ; sem <code>NetworkPolicy</code> com Calico habilitado; sem <code>PodDisruptionBudget</code>	Endurecer pods e rede
RSK-15	Baixo	Itens do código fora do forecast (\$11.5)	Incluir ou declarar
RSK-16	Baixo	Escalation Policy com 1 usuário	Escala com ≥ 2 pessoas
RSK-17	Baixo	<code>argocd_github_token</code> é obrigatória em <code>variables.tf</code> , mas não é usada em nenhum <code>.tf</code>	Remover ou tornar opcional
RSK-18	Baixo	O enunciado pede vídeo de até 20 min com pitch de 15 a 20 min mais a demo; o roteiro 05 prevê 17 a 18 min de fala, com 2 a 3 min de margem	Confirmar com a coordenação (Q-03)

SEÇÃO 20

Dificuldades e Lições Aprendidas

20.1 Problemas Corrigidos na Fase 5 (código e Git)

#	Problema	Correção	Evidência
1	Velero com <code>BackupStorageLocation</code> indisponível	Placeholders do <code>values.yaml</code> substituídos	[GIT] 5b4fa3b
2	<code>AuthorizationFailed</code> no plugin do Velerio	Role <code>Storage Account Contributor</code> (o plugin chama <code>listKeys</code>)	<code>velero.tf</code> ; [GIT] 3d5258e
3	Dashboard SRE com "No data" sem erros	<code>or vector(0)</code>	[GIT] 0431b1c
4	Datasources do Grafana desregistrados	Grafana fixado em 11.4.0	<code>values.yaml</code> ; [GIT] 742129c
5	404 nas rotas	Ingress sem <code>rewrite-target</code>	<code>ingress.yaml</code> ; [GIT] 1e5fcc5
6	Caracteres especiais nas credenciais das URLs de banco	<code>urlencode()</code> no usuário e na senha	<code>outputs.tf</code> ; [GIT] 649b89d , 6de6ea9
7	PAT reaproveitado da Fase 4 expirou ou foi revogado (comentário do <code>argocd.tf</code>)	Repositório GitOps público, clonado sem credencial	<code>argocd.tf</code> ; [GIT] 87cdad5
8	Troca da conta Datadog	Collector reapontado; chave atualizada diretamente no Git	[GIT] 35558e2 , 9ce7b3e
9	Flask não subia (<code>url_quote</code>)	<code>Werkzeug==2.2.3</code>	READMEs de ngo e volunteer

20.2 Lições da Fase 4 Registradas no Código

Lição (comentário no código)	Arquivo
Mínimo de 3 nós B2s	<code>aks.tf</code>
Grafana com estratégia <code>Recreate</code> (PVC RWO) e mais memória	<code>kube-prometheus-stack/values.yaml</code>
<code>domain</code> do Grafana e ingress sem <code>rewrite</code> (loop de redirect)	<code>values.yaml</code> ; <code>grafana-ingress.yaml</code>
Loki: <code>delete_request_store</code> ; <code>chunksCache</code> desligado	<code>loki/values.yaml</code>
<code>fullnameOverride: otel-collector</code> (DNS)	<code>otel-collector-gateway/values.yaml</code>
Métrica <code>trace.http.server.request.*</code>	<code>datadog.tf</code>
SP do self-heal criado pelo Terraform	<code>github_actions.tf</code>
Gunicorn sem <code>--preload</code>	<code>Dockerfile</code> do ngo e do volunteer

20.3 Lições Extraídas das Evidências [ANÁLISE]

Leitura desta especificação sobre fatos comprovados, com a fonte de cada fato entre parênteses.

- **Segredos:** a sequência `225bf01` (`ignoreDifferences`) → `b7b9ad7` (valores reais no Git) → `9ce7b3e` (Git como fonte da verdade para a chave do Datadog) mostra que, sem um mecanismo de segredos no repositório, os valores reais acabaram versionados [\[GIT\]](#).

- **SLI:** a instrumentação cobre todas as rotas; sem filtro de rota, o health check entra no SLI (`otel.go` , `deployment.yaml`).
- **Alerta:** o limiar do Monitor (5% em 5 min) detecta degradação aguda, mas não o consumo gradual do Error Budget de 30 d (`datadog.tf` , dashboard SRE).
- **Evidência:** a regra do enunciado prevê dedução para o que não for demonstrado (p. 6), e a pasta não tem prints nem vídeo.
- **Escopo do PCN:** a configuração mantém os backups na mesma região, e o documento 03 §1 declara esse limite.

SEÇÃO 21

Questões em Aberto

ID	Questão	Bloqueia
Q-01	Nomes, RMs e usernames dos integrantes — resolvida na versão 1.3 (capa e §1.1)	—
Q-02	Link do vídeo	DEL-06
Q-03	Os 15 a 20 min do pitch incluem a demo técnica?	DEL-04
Q-04	Valor real de <code>monitored_services</code> no <code>terraform.tfvars</code> de produção (arquivo ignorado pelo Git)	REQ-F3.2
Q-05	Manter a perda de região fora do PCN ou implementar backup em outra região?	REQ-F4.1
Q-06	Cláusula de compensação do SLA ("a formalizar comercialmente", documento 01 §5)	RN-04
Q-07	Os repositórios <code>freitasleoalves/fiap-tc-5-*</code> são os links oficiais da entrega?	DEL-06

SEÇÃO 22

Conclusão

O código da pasta cobre as cinco frentes do enunciado, com as ressalvas das seções 9 e 19:

- **Fundação:** Terraform, AKS, pipelines com testes, SAST e SCA, GitOps com ArgoCD, e observabilidade com Prometheus, Grafana, Loki, OpenTelemetry e Datadog.
- **SRE:** dois SLIs do caminho crítico com SLO e Error Budget no dashboard.
- **FinOps:** tags obrigatórias em código e Cosmos DB Serverless; o forecast está no documento 02.
- **ITSM:** Monitors ligados a PagerDuty, Discord e self-healing; o Watchdog não tem configuração em código.
- **DR:** PITR e Velero agendado, com metas de RPO/RTO registradas.

As lacunas que restam são de **evidência** (prints, vídeo, medição de MTTR e Watchdog), de **segurança** (RSK-01, RSK-03, RSK-12, RSK-13) e de **cobertura** (DR regional, Sev2 e SLI sem filtro de rota).

22.1 Resultado Quantitativo (código)

Indicador	Valor no código	Fonte
SLO de sucesso / latência	99,9% (30 d) / p95 ≤ 500 ms	Dashboard SRE; <code>datadog.tf</code>
Metas de RPO / RTO das doações	≤ 5 min / ≤ 30 min	<code>terraform/README.md</code>
Backup do cluster	Horário (<code>donation</code> , TTL 7 d) + diário (4 namespaces, TTL 30 d)	<code>schedule.yaml</code>
Retenção de backup do PostgreSQL	14 d (donation) / 7 d (ngo)	<code>postgresql.tf</code>
Recursos com tags FinOps	9	<code>*.tf</code>
Réplicas do caminho crítico	2 → 10 (HPA, CPU 70%)	<code>hpa.yaml</code>
Nós do cluster	3 → 5 (<code>Standard_B2s</code>)	<code>variables.tf</code>
Monitors por serviço monitorado	2 (erro 5xx e latência p95)	<code>datadog.tf</code>
Camadas no pipeline	Lint, testes, SAST, SCA (FS) e scan de imagem	<code>build-push.yaml</code>
Testes automatizados	14	<code>handlers_test.go</code> ; <code>test_app.py</code>

22.2 Links do Projeto

- <https://github.com/freitasleoalves/fiap-tc-5-terraform>
- <https://github.com/freitasleoalves/fiap-tc-5-gitops>
- <https://github.com/freitasleoalves/fiap-tc-5-donation-service>
- <https://github.com/freitasleoalves/fiap-tc-5-ngo-service>
- <https://github.com/freitasleoalves/fiap-tc-5-volunteer-service>
- <https://github.com/freitasleoalves/fiap-tc-5-notification-service>
- Vídeo: **[INSERIR LINK DO VÍDEO]**

ANEXO A

Mapeamento às Frentes e Escolhas de Ferramenta

A.1 Frentes → Artefatos

Frente	[ENUNCIADO]	Aplicação
0 — Fundação		§9.1; §5; <code>fiap-tc-5-terraform</code> ; <code>fiap-tc-5-gitops</code> ; <code>build-push.yaml</code>
1 — SRE		§10; 01; dashboard SRE; BR-01/BR-02
2 — FinOps		§11; 02; <code>main.tf</code> ; <code>cosmosdb.tf</code>
3 — ITSM/AIOps		§12; 04; <code>datadog.tf</code> ; <code>pagerduty.tf</code>
4 — Multicloud, Segurança e DR		§13; 03; Velero

A.2 Escolhas entre Alternativas do Enunciado

Decisão	Opções [ENUNCIADO]	Escolha no código	Justificativa registrada no código
Kubernetes	EKS, AKS, GKE	AKS	Não registrada; os READMEs citam continuidade dos padrões das Fases 3/4
Pipeline	GitHub Actions, GitLab CI	GitHub Actions	Não registrada
SAST / SCA	Trivy, Sonar	Os dois	<code>sonarqube/values.yaml</code> : Trivy cobre o SCA (dependências + imagem) e o SonarQube cobre o SAST
GitOps	ArgoCD, FluxCD	ArgoCD	Mesmo padrão App-of-Apps das Fases 3/4 (<code>fiap-tc-5-gitops/README.md</code>)
Observabilidade	Prometheus, Grafana, Loki e/ou OTel	Os quatro	Collector como peça central (<code>otel-collector-gateway/values.yaml</code>)
APM	Datadog, New Relic	Datadog	Comparação não registrada; configuração do exporter justificada em <code>values.yaml</code>
AIOps	Watchdog, Applied Intelligence	Watchdog	Comentário do <code>datadog.tf</code> : não há resource para ativá-lo; é automático assim que há dados de APM ou de infraestrutura na conta
Dashboard SRE	Grafana ou APM	Grafana	Prometheus com retenção de 30 d para calcular o Error Budget sem depender só do Datadog, pago por volume (<code>kube-prometheus-stack/values.yaml</code>)
DR	A: Velero · B: espelho em outra região	A	Não registrada no código (justificativa no documento 03 §7)
Mensageria / NoSQL	Não definidas no enunciado	Service Bus / Cosmos DB Table API	Substituem o SQS e o DynamoDB do código original (comentários em <code>servicebus.tf</code> e <code>cosmosdb.tf</code>); o toggle <code>CLOUD_PROVIDER</code> preserva o caminho AWS

ANEXO B

Auditoria de Aderência ao Código

Cópia dos documentos antes da auditoria: `C:\Users\demarchi\Desktop\fiap\_backup-docs-antes-auditoria-2026-09-25\`.

B.1 Correções Aplicadas nos Documentos 01 a 05

Doc	Trecho anterior	Correção	Evidência no código
01 §1	Referência externa ("Google SRE Book"); saturação "CPU/memória vs. requests/limits"	Referência removida; saturação só de CPU vs. requests; nota de que as métricas incluem <code>/health</code>	Painel Rightsizing; <code>otel.go</code> ; <code>main.go</code> ; <code>deployment.yaml</code>
01 §2	"PromQL, a mesma usada ... no Datadog Monitor"	Nota: o Monitor usa <code>trace.http.server.request.*</code> com limiar de 5% em 5 min	<code>datadog.tf</code>
01 §6	"self-healing via GitOps"; "Monitor avaliando SLIs"	GitHub Actions + <code>rollout restart</code> pelo Monitor de 5xx; nota de que os tempos são da Fase 4	<code>self-heal.yml</code> ; <code>datadog.tf</code>
02 cabeçalho	"Todos os valores" da API; região "em providers.tf"	O LB é valor de lista; região em <code>variables.tf</code>	<code>variables.tf</code> ; 02 §2
02 §1	"Toda a infraestrutura, sem exceção"; <code>Component = "hot-path-db"</code> ; LBs e discos do AKS listados como taguados	9 recursos com tags; <code>Component = "donation-service" + Criticality = "hot-path"</code> ; LBs e discos do AKS sem tags	<code>*.tf</code> ; <code>postgresql.tf</code>
02 §2	~40 GB de PVC; LB sem o ArgoCD; Velero em "Blob Cool/Hot"	32 Gi declarados; itens fora da tabela; Standard LRS	values dos addons; <code>argocd.tf</code> ; <code>velero.tf</code>
02 §3	"remoção do throughput = 400"; "o custo caiu"; "validado ... nesta sessão"	Serverless desde o commit <code>82421a8</code> ; diferença estimada; sem a frase de validação	<code>cosmosdb.tf</code> ; [GIT]
02 §4–§6	30–40% como fato; decisão "da Fase 4"; tabela "Antes/Depois"; "não um valor genérico copiado"	Faixa estimada; tabela "provisionado (estimado) × serverless (código)"; valores reais (ngo = notification); painel só de CPU	<code>deployment.yaml</code> ; dashboard Overview
03 §1	"falha de região" coberta; "decisão confirmada com o usuário"	Limite regional declarado; frase removida	<code>velero.tf</code> ; <code>postgresql.tf</code> ; <code>cosmosdb.tf</code>
03 §2	Nota "propostos por mim (Claude)"	Nota neutra, com referência ao README e aos comentários	<code>terraform/README.md</code>
03 §3	<code>ngo_db</code> "diário"; Cosmos "periódico... contínuo"; "entregue via ArgoCD"; "a zero no dado transacional"; role <code>Storage Blob Data Contributor</code>	Texto alinhado ao código; Schedules aplicados via <code>kubectl</code> ; RPO ≤ 5 min; <code>Storage Account Contributor</code>	<code>postgresql.tf</code> ; <code>cosmosdb.tf</code> ; <code>velero.tf</code>
03 §5	<code>application donation</code> ; o bootstrap "reconstrói ArgoCD + addons"	<code>app-donation</code> ; o ArgoCD vem do <code>argocd.tf</code>	<code>apps-appset.yaml</code> ; <code>argocd.tf</code>

Doc	Trecho anterior	Correção	Evidência no código
03 §7	Opção B descrita como "multi-região ativo-ativo"; "atendem com folga"	Opção B = espelho ativo-passivo em outra região (enunciado); tempo de restore ainda não medido	[ENUNCIADO] p. 5; 03 §4
04 §1–2	O Monitor "avalia a query do SLI"; self-heal em qualquer Monitor	Condições de <code>datadog.tf</code> ; self-heal só no Monitor de 5xx; nota sobre Sev2	<code>datadog.tf</code> ; <code>variables.tf</code>
04 §3	Ativação "ao habilitar Watchdog Insights"; propagação de trace "em todos os 4 serviços", inclusive "via HTTP"; U+200B	Texto de <code>datadog.tf</code> ; propagação só <code>donation</code> → <code>notification</code> pela fila; caractere removido	<code>datadog.tf</code> ; <code>servicebus.go</code> ; <code>app.py</code>
04 §4	"allowlist"; "decisão do usuário"	Removidos; <code>SERVICE_NAME</code> / <code>NAMESPACE</code> fixos	<code>self-heal.yml</code>
05	"números reais e não promessas"; ONGs e voluntários "informados em tempo real"; "sem custo desperdiçado"; 10 h atribuídas ao "processamento de doações"; "validei ao vivo, no ambiente real"; "qualquer anomalia"; "100% dos recursos"; "menos que uma doação média"; "Migramos"; "sem perder performance"; "desde o primeiro commit"; "nenhuma imagem com falha crítica"; "cada uma dessas promessas testada"; "três porquês" (só dois listados); caminhos <code>docs/</code>	Texto alinhado ao código: evento registrado; piso de 2 réplicas e 3 nós; cenário de referência da fase anterior; erro e latência; 9 recursos com tags; Serverless no código (estimativa); CVE crítica com correção disponível ; backup e restore executados; caminhos corrigidos	<code>notification-service/app.py</code> ; <code>hpa.yml</code> ; <code>aks.tf</code> ; <code>*.tf</code> ; <code>cosmosdb.tf</code> ; <code>build-push.yml</code> ; 03 §4
02, 03, 04	Links <code>../fiap-tc-5-...</code>	Links relativos à pasta <code>Entrega/</code>	Verificado: todos resolvem

B.2 Divergências no Próprio Código (não editadas)

O código pertence aos repositórios do grupo e não foi alterado. Estas divergências entre comentários ou README e o estado real ficam registradas para correção:

#	Local	Afirmiação	Estado real
1	<code>fiap-tc-5-gitops/README.md</code> (seção de secrets)	Todos os <code>secrets.yml</code> têm placeholders	Todos com valores reais (RSK-01)
2	<code>apps-appset.yml</code> ; <code>addons/otel-collector-gateway/application.yml</code> ("nunca commitamos a API Key real"); <code>addons/velero/application.yml</code>	Os <code>secrets.yml</code> versionados só têm placeholders	Valores reais desde <code>b7b9ad7</code>
3	<code>addons/velero/application.yml</code>	<code>include</code> aceita lista separada por vírgula	O documento 03 §4 relata que o <code>schedule.yml</code> não foi aplicado
4	<code>donation-service/.github/workflows/self-heal.yml</code> (cabeçalho)	Disparado pelo alerta de 5xx ou de latência	Só o Monitor de 5xx tem o webhook de self-heal
5	<code>datadog.tf</code> (mensagem do Monitor de 5xx)	> 5% em 5 min "viola o SLO de disponibilidade (99.9%)"	O SLO é medido em 30 dias
6	<code>apps/ngo/overlays/prod/ingress.yml</code> (comentário)	Cita a rota <code>/ngos/<id></code>	<code>app.py</code> só tem <code>/ngos</code> (GET/POST) e <code>/health</code>

#	Local	Afirmação	Estado real
7	<code>addons/velero/values.yaml</code>	"O Schedule roda o backup diariamente"; "ATENÇÃO: preencha com os valores reais"	Há dois Schedules (diário + horário); os valores já foram preenchidos (<code>5b4fa3b</code>)
8	<code>terraform/README.md</code> ; <code>gitops/README.md</code>	Remetem a "Fase C do plano de execução" / <code>CONTEXT0-FASE5.md</code>	Documentos ausentes na pasta
9	<code>variables.tf</code>	<code>argocd_github_token</code> obrigatória	Nenhum <code>.tf</code> usa a variável
10	<code>kube-prometheus-stack/values.yaml</code>	Custo extra da retenção de 30 d "documentado no relatório de Forecast"	O <code>02</code> só tem uma linha agregada de PVC
11	<code>main.tf</code> (comentário); <code>terraform/README.md</code>	Tags "aplicadas em 100% dos recursos de nuvem"	9 recursos declaram <code>tags</code> ; subnet, bancos, regras de firewall, fila, tabelas, container, role assignments e node pool do AKS não declaram
12	Dashboard SRE (painel de texto "SLO de Latência")	"p95 ≤ 500ms em <code>POST/GET /donations</code> "	A consulta do painel p95 filtra só <code>service_name</code> : mede todas as rotas, inclusive <code>/health</code> (RSK-08)

ANEXO C

Inventário de Artefatos Analisados

Artefato	HEAD	Commits	Data do HEAD
fiap-tc-5-terraform	3d5258e	5	2026-09-25
fiap-tc-5-gitops	4b19ac5	16	2026-09-25
fiap-tc-5-donation-service	09f6e6e	3	2026-09-25
fiap-tc-5-ngo-service	73d5d36	2	2026-09-10
fiap-tc-5-volunteer-service	87ae20a	2	2026-09-10
fiap-tc-5-notification-service	5fafa72	1	2026-09-10
Documentos 01 a 05	Revisados na versão 1.2	—	2026-09-25
Enunciado POSTECH - DCLT - Hackathon - Fase 5	8 páginas	—	—

ANEXO D

Versões de Componentes

Componente	Versão	Fonte
Terraform	≥ 1.5.0	<code>providers.tf</code>
Providers	azurerem ~> 5.5 · azuread ~> 3.9 · helm ~> 3.3 · datadog ~> 4.21 · pagerduty ~> 3.36	<code>providers.tf</code>
ArgoCD · ingress-nginx (charts)	9.4.15 · 4.12.1	<code>argocd.tf</code>
kube-prometheus-stack · Grafana	90.0.0 · 11.4.0	<code>addons/kube-prometheus-stack/</code>
Loki	7.3.0	<code>addons/loki/application.yaml</code>
OpenTelemetry Collector	chart 0.173.0 · imagem 0.160.0	<code>addons/otel-collector-*/</code>
Velero · plugin Azure	12.1.0 · v1.13.1	<code>addons/velero/</code>
SonarQube	2026.4.1	<code>addons/sonarqube/application.yaml</code>
PostgreSQL	15	<code>postgresql.tf</code>
Go · OpenTelemetry Go	1.25 · 1.46.0	<code>go.mod</code>
Python · Flask · Werkzeug · gunicorn · OTel distro	3.11 · 2.2.2 · 2.2.3 · 20.1.0 · 0.62b1	<code>requirements.txt</code>

ANEXO E

Glossário

Termo	Definição	Fonte
SLI	Indicadores do dashboard SRE: taxa de sucesso (não-5xx) e latência p95 do <code>donation-service</code>	<code>solidarytech-sre-dashboard.yaml</code>
SLO	Metas dos painéis: 99,9% de sucesso em 30 d e $p95 \leq 500$ ms	Dashboard SRE; <code>datadog.tf</code>
SLA	Compromisso externo com as ONGs, definido no documento 01 §5	Documento 01 §5
Error Budget	Parcela de falhas permitida pelo SLO (0,1%); o gauge divide a taxa de erro por <code>0.001</code>	Dashboard SRE
Golden Metrics	Latência, tráfego, erros e saturação; mapeadas às métricas do código no documento 01 §1	Documento 01 §1
MTTR	Mean Time To Recovery	[ENUNCIADO] p. 4
RPO / RTO	Metas de perda de dados e de tempo de recuperação, registradas no README do Terraform	<code>terraform/README.md</code>
PITR	Point-in-Time Restore do PostgreSQL, habilitado pela retenção de backup	<code>postgresql.tf</code>
Hot Path	Caminho Crítico: o processamento de doações	[ENUNCIADO] p. 2
PCN	Plano de Continuidade de Negócios	[ENUNCIADO] p. 5
GitOps	O repositório define o estado desejado; o ArgoCD monitora o repositório e sincroniza o cluster	<code>fiap-tc-5-gitops/README.md</code>
App-of-Apps / ApplicationSet	Ponto de entrada do ArgoCD / gerador de Applications a partir de <code>apps/*/overlays/prod</code>	<code>fiap-tc-5-gitops/README.md</code>
HPA	HorizontalPodAutoscaler do <code>donation-service</code>	<code>hpa.yaml</code>
Rightsizing	Ajuste de requests/limits a partir das métricas	[ENUNCIADO] p. 4
Watchdog	AIOPS do Datadog; segundo o comentário do código, automático assim que há dados de APM ou de infraestrutura na conta	<code>datadog.tf</code>
SAST / SCA	Análise estática de código (SonarQube) / de dependências e imagem (Trivy)	<code>sonarqube/values.yaml</code>
Velero	Backup do estado do cluster para fora do cluster: manifestos no Azure Blob Storage e volumes por snapshot de Azure Disk	<code>addons/velero/values.yaml</code> ; <code>velero.tf</code>
Warm Standby	Ambiente espelho em outra região (Opção B)	[ENUNCIADO] p. 5